

神经网络基础

主要内容

- 多层感知器模型
- 卷积神经网络
- 循环神经网络
- 以Transformer为代表的自注意力模型
- 每种模型的PyTorch调用方式
- 情感分类为代表的文本分类任务
- 词性标注为代表的序列标注任务

感知器模型

感知器 (Perceptron)

- 最简单也是最早出现的机器学习模型
- 灵感直接来源于生产生活的实践

面试评分

- $S = x_1 + x_2 + x_3 + x_4$
 - 如果 $s \geq t$ 则录用, 否则不录用
- 考虑面试官的经验(权重)
 - $S = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4$

感知器模型

$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b \geq 0 \\ 0 & \text{else} \end{cases}, \text{ 其中 } b = -t$$



感知器模型

待解决问题：

□ 特征提取

□ 原始输入 $\rightarrow$ 向量 x

□ 文本表示为向量

□ 参数学习

□ 权重 w 和偏差项 b

应用：用户评论的情感极性，分类问题

机器学习模型

模型	输出结果	典型的模型	应用
分类模型	离散类别	感知器	☐ 用户句子评论(褒义和贬义)
回归模型	连续的实数值	☐ 线性回归 ☐ Logistic回归 ☐ Softmax回归	☐ 预测股票的指数 ☐ 天气预报中温度的预测 ☐ 电商(电影)评分

▣ 线性回归 (Linear Regression)

▣ 形式: $y = \sum_i w_i x_i + b$

▣ 值域: $\mathbb{R}$

▣ 示意图:

多层感知器模型-Logistic回归模型

□ Logistic回归 (Logistic Regression)

□ Logistic函数

$$y = \frac{L}{1 + e^{-k(z-z_0)}} \quad z = \sum_i w_i x_i + b$$

□ 值域 : (0 , L)

$$z \rightarrow -\infty \quad y \rightarrow 0$$

$$z \rightarrow \infty \quad y \rightarrow L$$

$$z = z_0 \quad y = L / 2$$

□ Sigmoid函数 (Logistic函数的特例)

$$y = \frac{1}{1 + e^{-z}} \quad L=1, \quad k=1, \quad z_0=0$$

多层感知器模型-Logistic回归模型

❑ Sigmoid函数₁

❑ 形式: $y = \frac{1}{1 + e^{-z}}$

❑ 图形:

❑ 值域: (0,1)

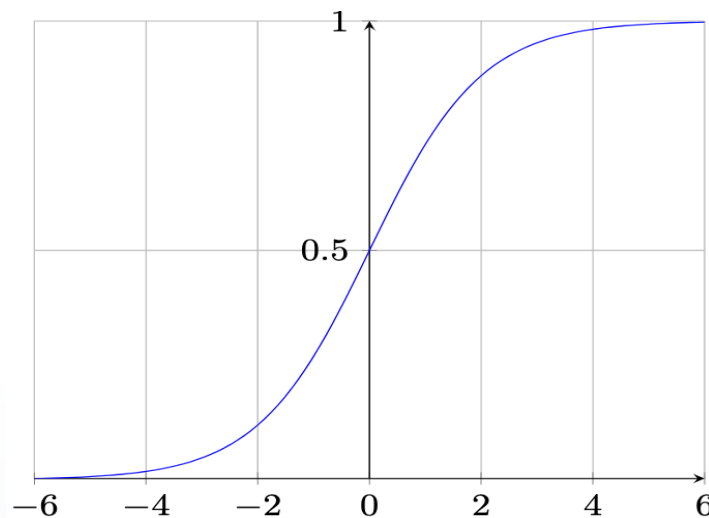
❑ 优点:

❑ 概率: 输入数据属于某个类别的概率

❑ 导数: 导数易求($y' = y(1-y)$), 为基于梯度的优化方法提供便利

❑ 应用: 处理二元分类问题, y 为输出的**概率**

❑ 垃圾邮件过滤、褒贬识别



Sigmoid函数示意图

多层感知器模型-Softmax回归

▣ 多分类问题:

▣ 手写数字识别、文本分类、词性标注、情感极性

▣ Softmax函数

$$y_i = \text{Softmax}(z)_i = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \dots + e^{z_m}}$$

▣ 其中 y_i 为第 i 个类别的概率, $z_i = \sum_j w_{ij}x_k + b$, w_{ij} 为第 i 个类别所对应的第 j 个输入的权重

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \text{Softmax} \begin{pmatrix} w_{11}x_1 + w_{12}x_2 + \dots + w_{1n}x_n + b_1 \\ w_{21}x_1 + w_{22}x_2 + \dots + w_{2n}x_n + b_2 \\ \vdots \\ w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mn}x_n + b_m \end{pmatrix}$$

多层感知器模型-Softmax回归

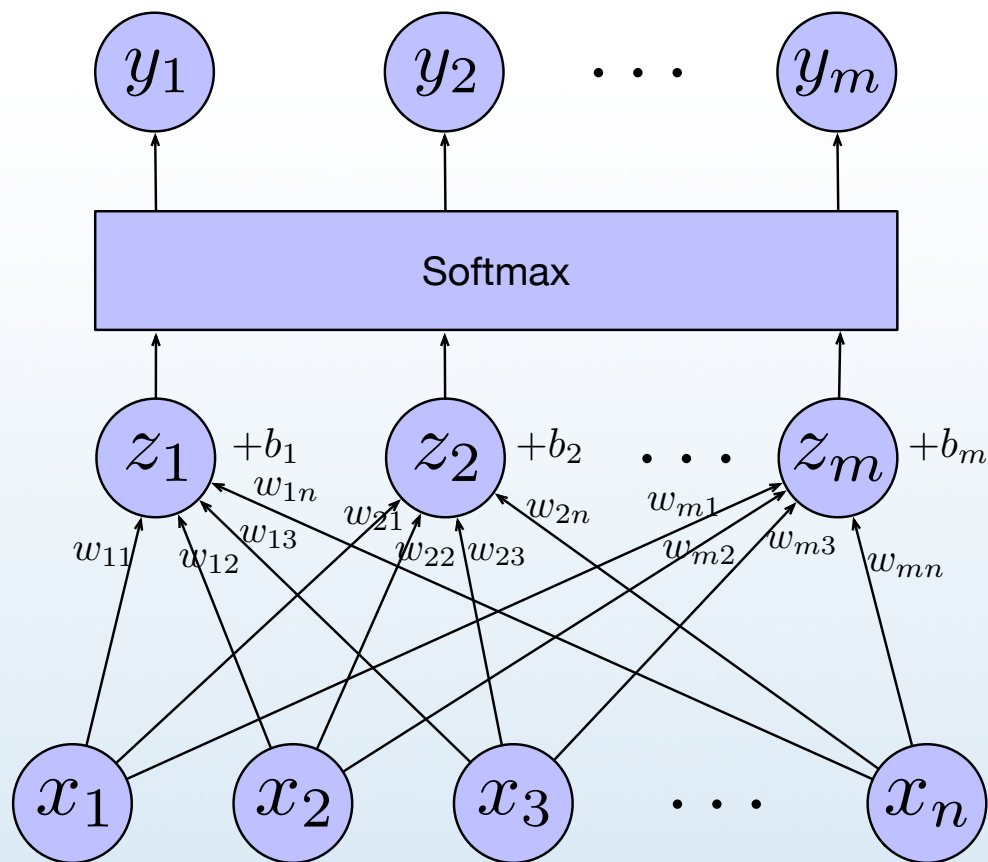
$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \text{Softmax} \begin{pmatrix} w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n + b_1 \\ w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n + b_2 \\ \vdots \\ w_{m1}x_1 + w_{m2}x_2 + \cdots + w_{mn}x_n + b_m \end{pmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \text{Softmax} \left(\begin{bmatrix} w_{11}, w_{12}, \cdots, w_{1n} \\ w_{21}, w_{22}, \cdots, w_{2n} \\ \vdots \\ w_{m1}, w_{m2}, \cdots, w_{mn} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \right)$$

$$\mathbf{y} = \text{Softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

多层感知器模型-Softmax回归

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \text{Softmax} \left(\begin{bmatrix} w_{11}, w_{12}, \dots, w_{1n} \\ w_{21}, w_{22}, \dots, w_{2n} \\ \vdots \\ w_{m1}, w_{m2}, \dots, w_{mn} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \right)$$

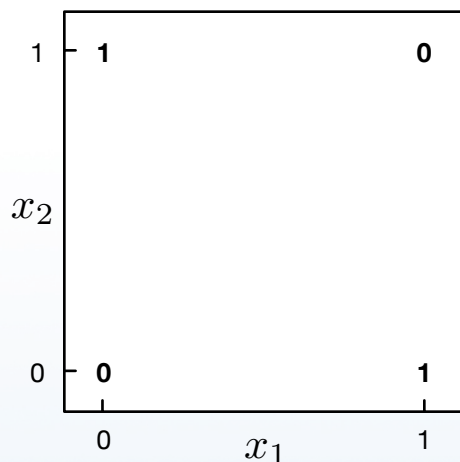


多层感知器

□问题：线性模型不能解决线性不可分问题

□例：异或问题

x_1	x_2	y
0	0	0
1	1	0
1	0	1
0	1	1



□解决：多层感知器（ Multi-layer Perceptron , MLP ）

□堆叠多层感知器（ 线性回归+非线性激活函数 ）

□可解决线性不可分问题

多层感知器

使用MLP解决异或 (XOR) 问题

$$z = W^{[1]}x + b^{[1]}$$

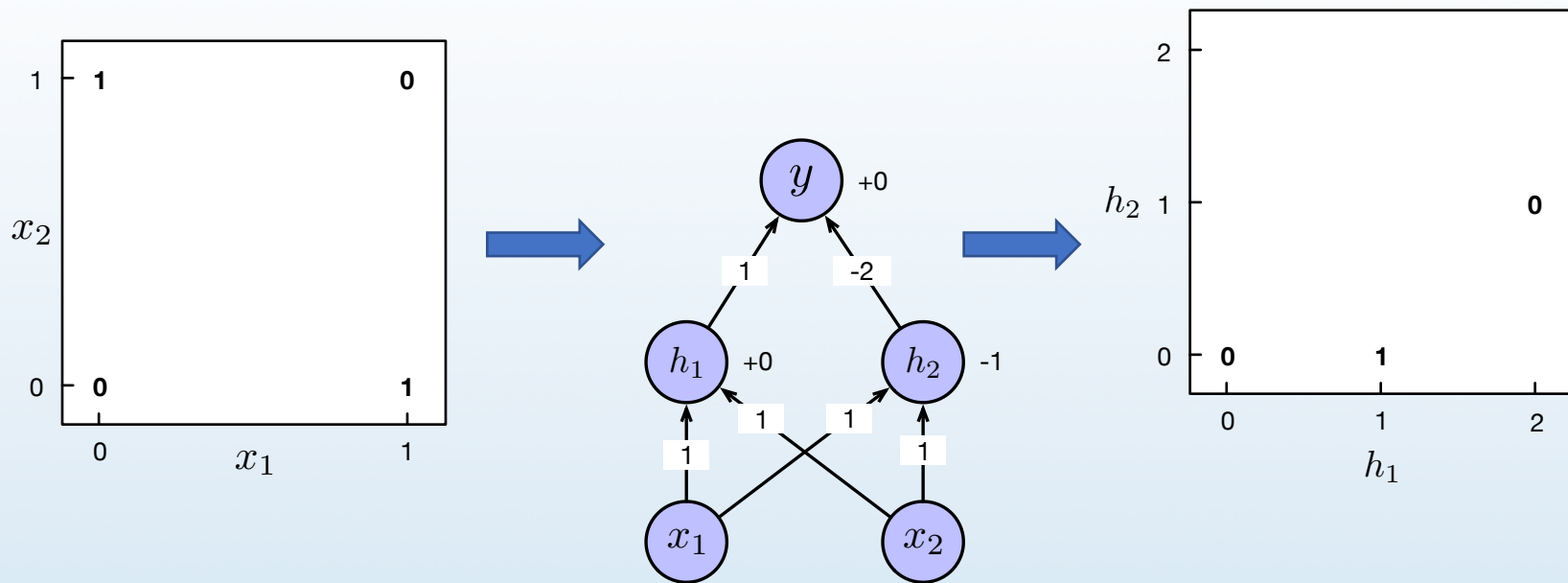
$$h = \text{ReLU}(z)$$

$$y = W^{[2]}h + b^{[2]}$$

$$W^{[1]} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad b^{[1]} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$$W^{[2]} = [1, -2] \quad b^{[2]} = [0]$$

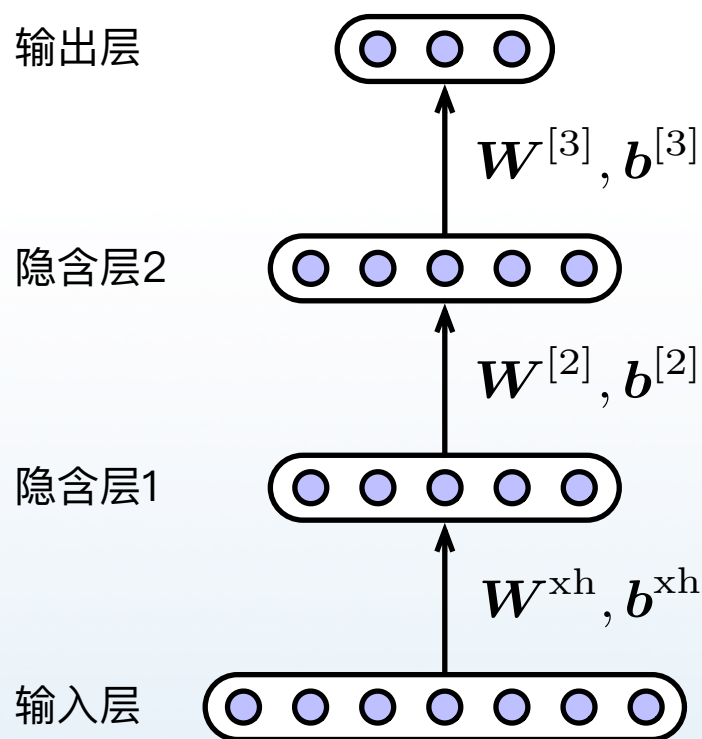
$$\text{ReLU}(z) = \max(0, z)$$



多层感知器

□ 多层感知器

- 输入层大小：输入数据维度
- 输出层大小：处理问题的类别
- 隐含层：大小、层数、激活函数跟经验相关
- 超参数：隐含层大小、层数、权重、偏置等



多层感知器模型-多层感知器PyTorch实现

□引入PyTorch模块

□>>> `from torch import nn`

□线性层

输入特征数目

输出特征数目

```
>>> linear = nn.Linear(in_features, out_features)
```

```
>>> linear = nn.Linear(32, 2) # 输入32维, 输出2维
>>> inputs = torch.rand(3, 32) # 创建一个形状为(3, 32)的随机张量, 3为批次大小
>>> outputs = linear(inputs) # 输出张量形状为(3, 2)
>>> print(outputs)
tensor([[ 0.5387, -0.4537],
        [ 0.2181, -0.3745],
        [ 0.3704, -0.8121]], grad_fn=<AddmmBackward>)
```

多层感知器模型-多层感知器PyTorch实现

□ 激活函数

□ >>> `from torch.nn import functional as F`

□ >>> Sigmoid、Softmax、ReLU 函数，tanh 等多种
激活函数等各种激活函数

```
>>> activation = F.sigmoid(outputs)
>>> print(activation)
tensor([[0.6315, 0.3885],
        [0.5543, 0.4075],
        [0.5916, 0.3074]], grad_fn=<SigmoidBackward>)
>>> activation = F.softmax(outputs, dim=1)
# 沿着第2维进行Softmax运算，即对每批次中的各样例分别进行Softmax运算
>>> print(activation)
tensor([[0.7296, 0.2704],
        [0.6440, 0.3560],
        [0.7654, 0.2346]], grad_fn=<SoftmaxBackward>)
>>> activation = F.relu(outputs)
>>> print(activation)
tensor([[0.5387, 0.0000],
        [0.2181, 0.0000],
        [0.3704, 0.0000]], grad_fn=<ReluBackward0>)
```

多层感知器模型-多层感知器PyTorch实现

自定义神经网络实现

构造函数定义了模型所需的成员对象，如构成该模型的各层，并对其中的参数进行初始化等

```
class MLP(nn.Module):  
    def __init__(self,  
        super(MLP, self).__init__()  
        # 线性变换：输入层->隐含层  
        self.linear1 = nn.Linear(input_dim, hidden_dim)  
        # 使用ReLU激活函数
```

```
mlp = MLP(input_dim=4, hidden_dim=5, num_class=2)
```

```
inputs = torch.rand(3, 4) # 输入形状为(3, 4)的张量，其中3表示有3个输入，4表示每个  
    输入的维度
```

实现该模块的前向过程，即对输入进行逐层的处理，从而得到最终的输出结果

forward函数

模型示意图？

输出的概率

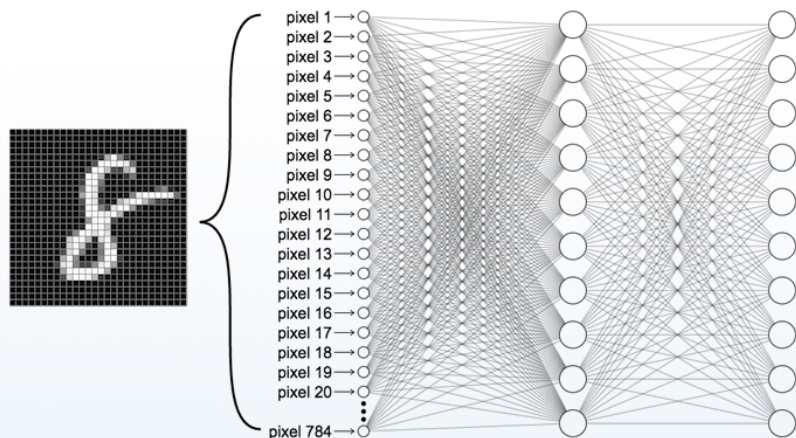
```
hidden = self.linear1(inputs)  
activation = self.activate(hidden)  
outputs = self.linear2(activation)  
probs = F.softmax(outputs, dim=1) # 获得每个输入属于某一类别的概率  
return probs
```

卷积神经网络

❑ 全连接的多层感知器无法处理输入的偏移情况

❑ MLP的每层输入的各个元素都需要乘以一个独立的参数

❑ 图像分类



每个像素赋予独立的参数，一旦待识别物体的位置出现轻微移动，识别结果可能会发生较大的变化

❑ 情感分类

❑ 我喜欢自然语言处理。 vs. 我非常喜欢自然语言处理。

❑ 情感极性往往由个别词或短语决定，而这些决定性的词或短语在句子中的位置并不固定，使用全连接层很难捕捉这种关键的局部信息

卷积神经网络

□ 解决方案

□ **想法**：使用一个小的稠密层提取这些局部特征，如

□ **图像**: 固定大小的像素区域 (16x16、8x8、...)

□ **文本**: 词的N-gram等(“我喜欢自然语言处理”)

□ **问题**：关键信息位置不固定

□ **解决**：依次扫描输入的每个区域，该操作被称为卷积(Con-volution)操作

□ **卷积核(Kernel)or滤波器(Filter)**：每个小的、用于提取局部特征的稠密层

卷积神经网络

卷积核构造

□卷积核的实际需求：

- 实际问题中，往往需要提取很多种局部特征

 - 如在情感分类中不同的情感词或者词组等

- 在进行卷积操作时

 - 若仅使用一个卷积核，只能提取单一类型的局部特征

 - 可以使用多个卷积核提取不同种类的局部特征

□卷积核的构造方式大致有两种

- ①使用不同组的参数，获得不同的卷积核

- ②提取不同尺度的局部特征

 - 如在情感分类中提取不同大小的N-gram

卷积神经网络

池化

- 池化操作：对卷积操作输出结果进行进一步聚合的过程
- 常用池化操作：最大池化、平均池化和加和池化等
 - 例如：最大池化仅保留最有意义的局部特征
 - 情感分类：保留句子中对于分类最关键的N-gram信息
- 池化操作的好处：解决样本的输入大小不一致的问题
 - 如对于情感分类
 - 有的句子比较长，有的句子比较短
 - 不同句子包含不同N-gram数目，局部特征个数不相同
 - 池化操作后，可保证最终输出相同个数的特征

决策

- 将多个卷积核输出的多个特征，经过一个全连接的分类层就可以做出最终的决策
- 还可以将多个卷积层加池化层堆叠起来，形成更深层的网络，这些网络统称为卷积神经网络（Convolutional Neural Network，CNN）

卷积神经网络

❑ 输入：“我喜欢自然语言处理。”

❑ 向量：维度为5

❑ 卷积：4个

❑ 宽度为4：

❑ 2个(黄、蓝)，输出维度3

❑ 宽度为3：

❑ 2个(绿、红)，输出维度4

❑ 每个卷积核的输出长度： $L - N + 1$

❑ L 为单词的个数

❑ N 为卷积核的宽度

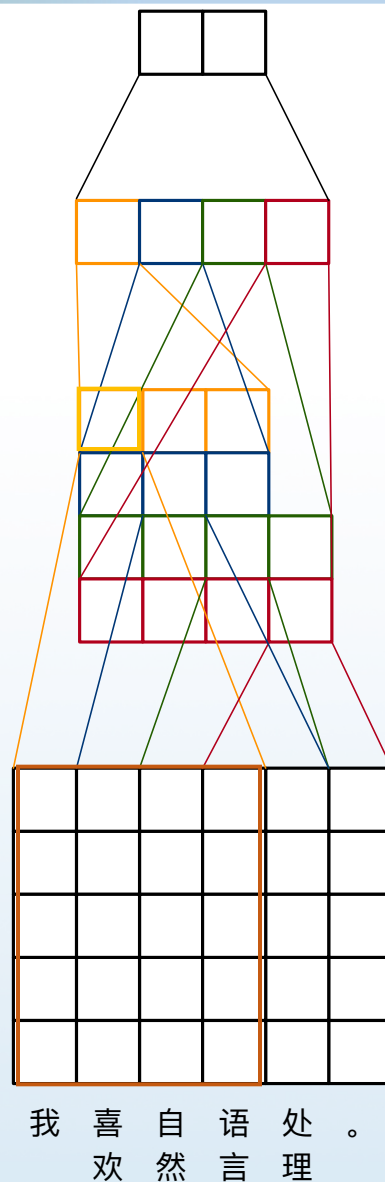
❑ 池化：最大

全连接层

池化层

卷积层

输入层



卷积神经网络

❑ 输入：“我喜欢自然语言处理。”

❑ 向量：维度为5

❑ 卷积：4个

❑ 宽度为4：

❑ 2个(黄、蓝)，输出维度3

❑ 宽度为3：

❑ 2个(绿、红)，输出维度4

❑ 每个卷积核的输出长度： $L - N + 1$

❑ L 为单词的个数

❑ N 为卷积核的宽度

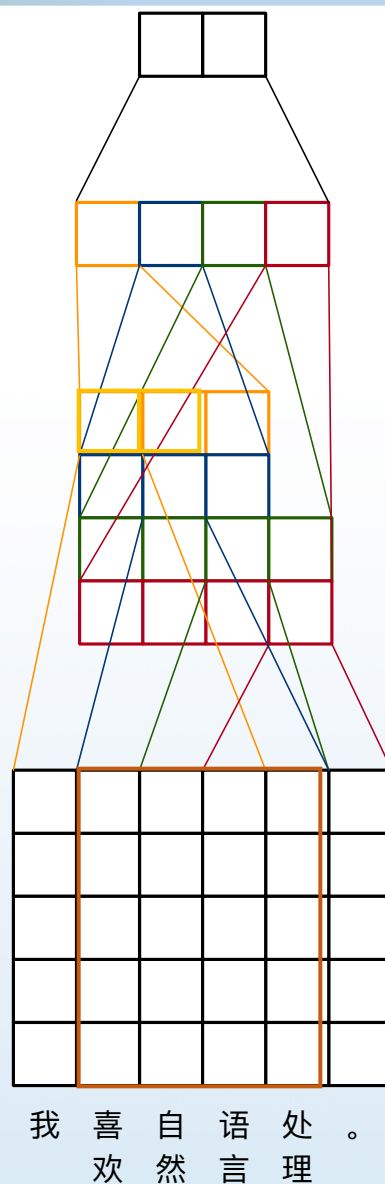
❑ 池化：最大

全连接层

池化层

卷积层

输入层



卷积神经网络

❑ 输入：“我喜欢自然语言处理。”

❑ 向量：维度为5

❑ 卷积：4个

❑ 宽度为4：

❑ 2个(黄、蓝)，输出维度3

❑ 宽度为3：

❑ 2个(绿、红)，输出维度4

❑ 每个卷积核的输出长度： $L - N + 1$

❑ L 为单词的个数

❑ N 为卷积核的宽度

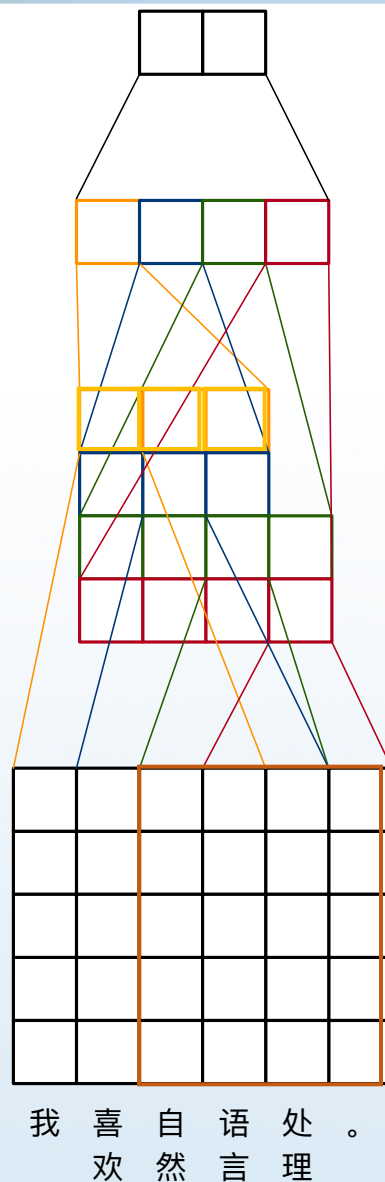
❑ 池化：最大

全连接层

池化层

卷积层

输入层



卷积神经网络

□ 输入：“我喜欢自然语言处理。”

□ 向量：维度维度为5

□ 卷积：4个

□ 宽度为4：

□ 2个(黄、蓝)，输出维度3

□ 宽度为3:

□ 2个(绿、红)，输出维度4

□ 每个卷积核的输出长度： $L - N + 1$

□ L 为单词的个数

□ N 为卷积核的宽度

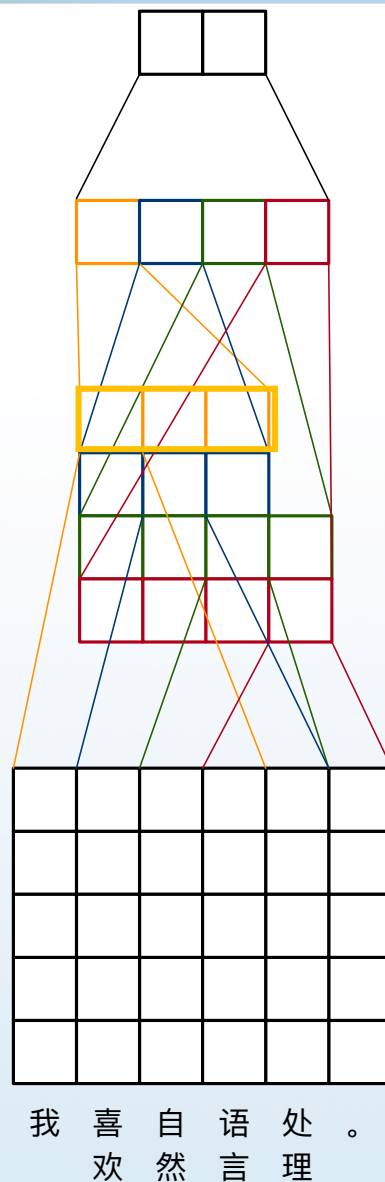
□ 池化：最大

全连接层

池化层

卷积层

输入层



卷积神经网络

❑ 输入：“我喜欢自然语言处理。”

❑ 向量：维度维度为5

❑ 卷积：4个

❑ 宽度为4：

❑ 2个(黄、蓝)，输出维度3

❑ 宽度为3:

❑ 2个(绿、红)，输出维度4

❑ 每个卷积核的输出长度： $L-N+1$

❑ L 为单词的个数

❑ N 为卷积核的宽度

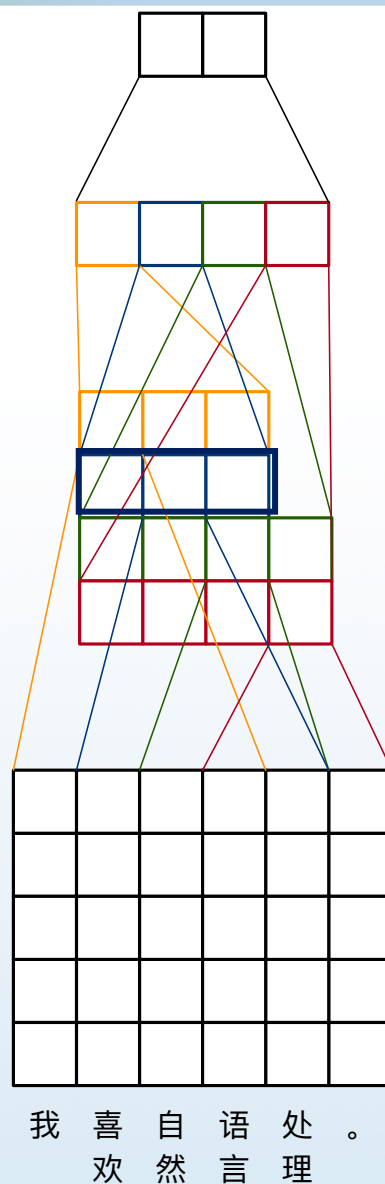
❑ 池化：最大

全连接层

池化层

卷积层

输入层



卷积神经网络

❑ 输入：“我喜欢自然语言处理。”

❑ 向量：维度为5

❑ 卷积：4个

❑ 宽度为4：

❑ 2个(黄、蓝)，输出维度3

❑ 宽度为3：

❑ 2个(绿、红)，输出维度4

❑ 每个卷积核的输出长度： $L - N + 1$

❑ L 为单词的个数

❑ N 为卷积核的宽度

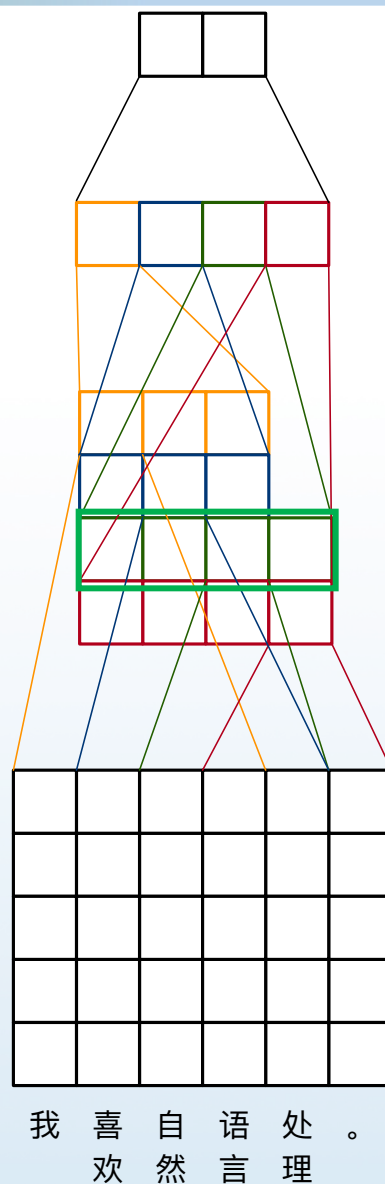
❑ 池化：最大

全连接层

池化层

卷积层

输入层



卷积神经网络

❑ 输入：“我喜欢自然语言处理。”

❑ 向量：维度维度为5

❑ 卷积：4个

❑ 宽度为4：

❑ 2个(黄、蓝)，输出维度3

❑ 宽度为3:

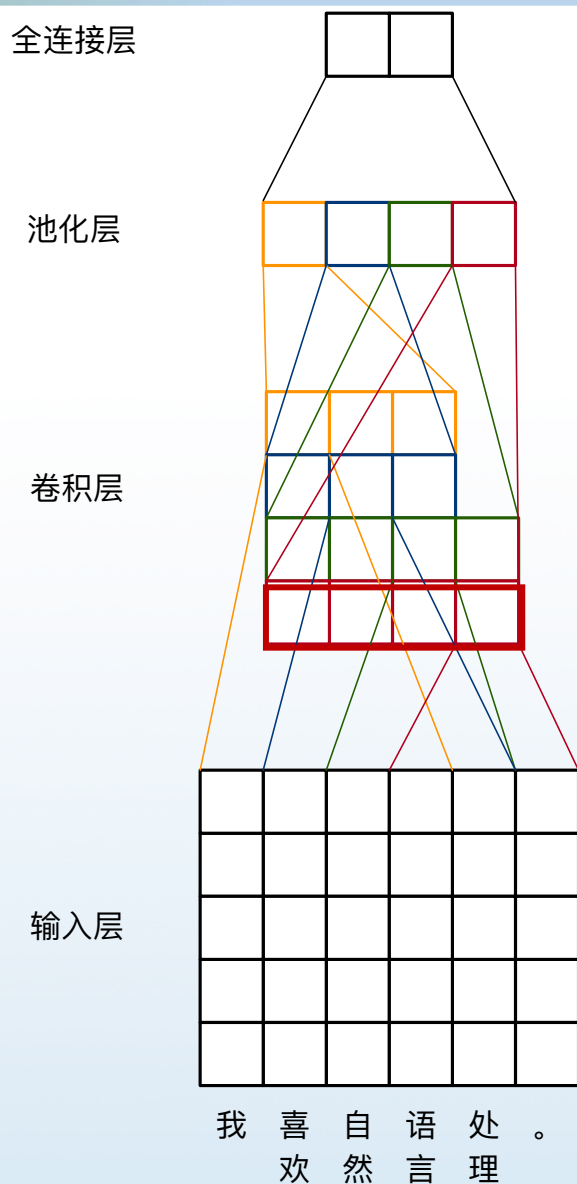
❑ 2个(绿、红)，输出维度4

❑ 每个卷积核的输出长度： $L - N + 1$

❑ L 为单词的个数

❑ N 为卷积核的宽度

❑ 池化：最大



卷积神经网络

□卷积：

- 一维卷积：卷积核沿单一方向滑动，适用于自然语言等序列数据
- 二维卷积：卷积核沿横向+纵向滑动，适用于图像数据
- 三维卷积：卷积核沿三个方向滑动，适用于视频数据

卷积神经网络

- 解决全连接的MLP无法处理输入的偏移情况
 - 使用小的全连接层抽取局部特征（卷积核或滤波器）
 - 使用多个卷积核提取不同种类的特征
 - 使用池化层将特征进行聚合（最大、平均、求和等）

卷积神经网络-PyTorch实现

- ❑ PyTorch的torch.nn包中使用Conv1d、Conv2d或Conv3d类实现卷积层，它们分别表示一维卷积、二维卷积和三维卷积
- ❑ NLP中常用的一维卷积（Conv1d）
 - ❑ in_channels: 输入通道的个数，在输入层对应词向量的维度
 - ❑ out_channels: 输出通道的个数，对应卷积核的个数
 - ❑ kernel_size: 每个卷积核的宽度
 - ❑ 输入数据形状为（batch，in_channels，seq_len）
 - ❑ 输出数据形状为（batch，out_channels，seq_len）
 - ❑ 在输入数据和输出数据中，seq_len分别表示输入的序列长度和输出的序列长度

```

>>> import torch
>>> from torch.nn import Conv1d
>>> conv1 = Conv1d(5, 2, 4)
    # 定义一个一维卷积，输入通道大小为5，输出通道大小为2，卷积核宽度为4
>>> conv2 = Conv1d(5, 2, 3) # 再定义一个一维卷积，输入通道大小为5，输出通道大小
    # 为2，卷积核宽度为3
>>> inputs = torch.rand(2, 5, 6) # 输入数据批次大小为2，即有两个序列，每个序列的
    # 长度为6，每个输入的维度为5
>>> outputs1 = conv1(inputs)
>>> outputs2 = conv2(inputs)
>>> print(outputs1) # 第1个输出为两个序列，每个序列长度为3，大小为2
tensor([[[ 0.2402,  0.1363,  0.1578],
          [ 0.2771, -0.0916, -0.3951]],

        [[ 0.3577,  0.2122,  0.2909],
          [-0.2675,  0.1801, -0.0385]]], grad_fn=<SqueezeBackward1>)
>>> print(outputs2) # 第2个输出也为两个序列，每个序列长度为4，大小为2
tensor([[[ 0.3900,  0.1210, -0.0137, -0.0562],
          [-0.5736, -0.5723, -0.4178, -0.3327]],

        [[ 0.2690,  0.3945,  0.2949,  0.0736],
          [-0.7219, -0.7087, -0.4591, -0.4186]]], grad_fn=<SqueezeBackward1>)

```

□池化

□池化层对象

- torch.nn包中定义的池化层类：最大池化、平均池化等

- 池化方法：一维、二维和三维三种

- 一维：MaxPool1d(kernel_size)

 - kernel_size:池化层大小

□池化函数

卷积神经网络-PyTorch实现

□池化

□池化层对象

□torch.nn包中定义的池化层类：最大池化、平均池化等

```
>>> from torch.nn import MaxPool1d
>>> pool1 = MaxPool1d(3) # 第1个池化层核的大小为3，即卷积层的输出序列长度
>>> pool2 = MaxPool1d(4) # 第2个池化层核的大小为4
>>> outputs_pool1 = pool1(outputs1)
# 执行一维最大池化操作，即取每行输入的最大值
```

```
>>> outputs_pool2 = pool1(outputs2)
>>> print(outputs_pool1)
```

```
tensor([[[0.2402],
          [0.2771]],
        [[0.3577],
          [0.1801]]], grad_fn=<SqueezeBackward1>)
```

```
>>> print(outputs_pool2)
tensor([[[ 0.3900],
          [-0.4178]],
        [[ 0.3945],
          [-0.4591]]], grad_fn=<SqueezeBackward1>)
```


卷积神经网络-PyTorch实现

□池化

□池化功能

```
>>> import torch.nn.functional as F
>>> outputs_pool1 = F.max_pool1d(outputs1, kernel_size=outputs1.shape[2])
# outputs1的最后一维恰好为其序列的长度
>>> print(outputs_pool1)
tensor([[[[0.2402],
          [0.2771]],

        [[0.3577],
          [0.1801]]], grad_fn=<SqueezeBackward1>)
>>> outputs_pool2 = F.max_pool1d(outputs2, kernel_size=outputs2.shape[2])
>>> print(outputs_pool2)
tensor([[[[ 0.3900],
          [-0.3327]],

        [[ 0.3945],
          [-0.4186]]], grad_fn=<SqueezeBackward1>)
```

七功

小，

适合

卷积神经网络-PyTorch实现

□ 拼接

□ 全连接层分类

卷积神经网络-PyTorch实现

□ 拼接

□ outputs_pool1和outputs_pool2是两个独立的张量

□ 调用squeeze函数将最后一个为1的维度删除，即

```
>>> outputs_pool_squeeze1 = outputs_pool1.squeeze(dim=2)
>>> print(outputs_pool_squeeze1)
tensor([[0.2402, 0.2771],
        [0.3577, 0.1801]], grad_fn=<SqueezeBackward1>)
>>> outputs_pool_squeeze2 = outputs_pool2.squeeze(dim=2)
>>> print(outputs_pool_squeeze2)
tensor([[ 0.3900, -0.3327],
        [ 0.3945, -0.4186]], grad_fn=<SqueezeBackward1>)
>>> outputs_pool = torch.cat([outputs_pool_squeeze1, outputs_pool_squeeze2],
                             dim=1)
>>> print(outputs_pool)
tensor([[ 0.2402,  0.2771,  0.3900, -0.3327],
        [ 0.3577,  0.1801,  0.3945, -0.4186]], grad_fn=<CatBackward>)
```

循环神经网络

❑ 卷积神经网络无法处理长距离的依赖

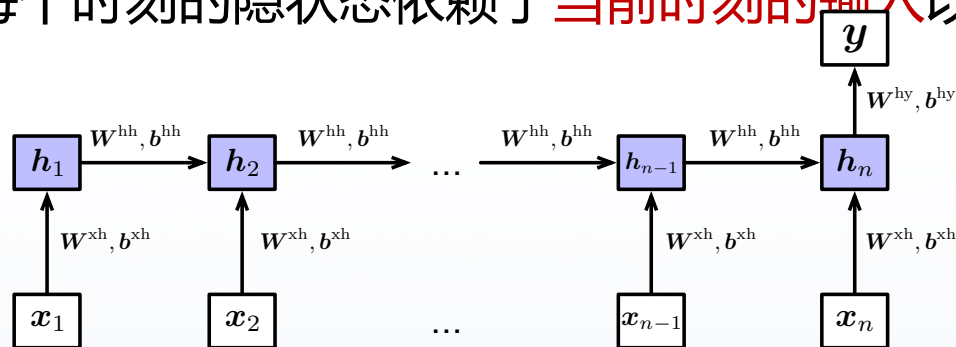
❑ 循环神经网络的解决方案

❑ 每个时刻的隐状态依赖于当前时刻的输入以及上一时刻的隐状态

输出层

隐含层

输入层



$$h_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$$

$$y = \text{Softmax}(W^{hy}h_t + b^{hy})$$

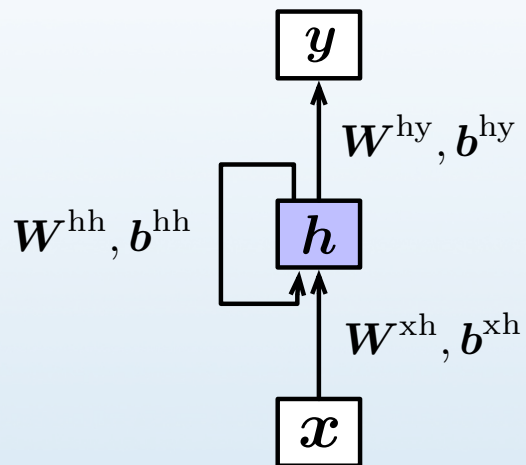
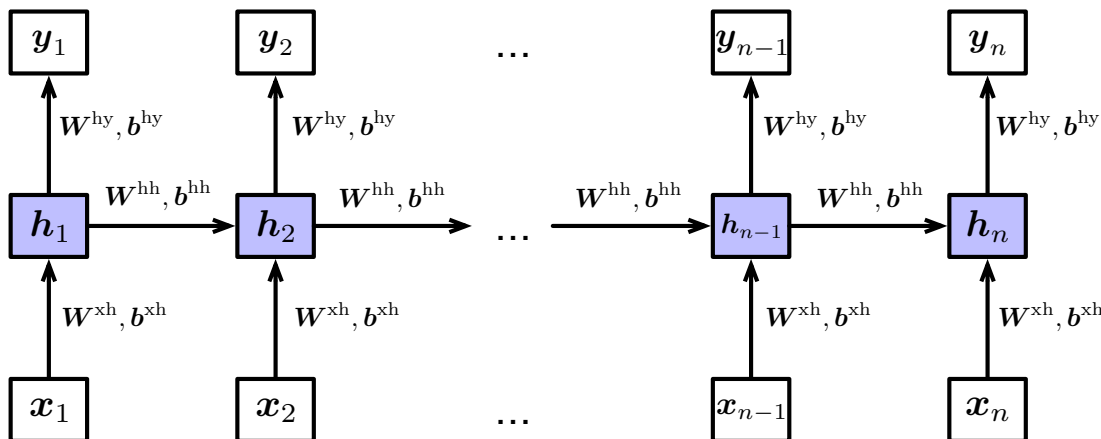
❑ 每个时刻的参数共享 (“循环” 的由来)

❑ 每个时刻也可以有相应的输出 (序列标注)

输出层

隐含层

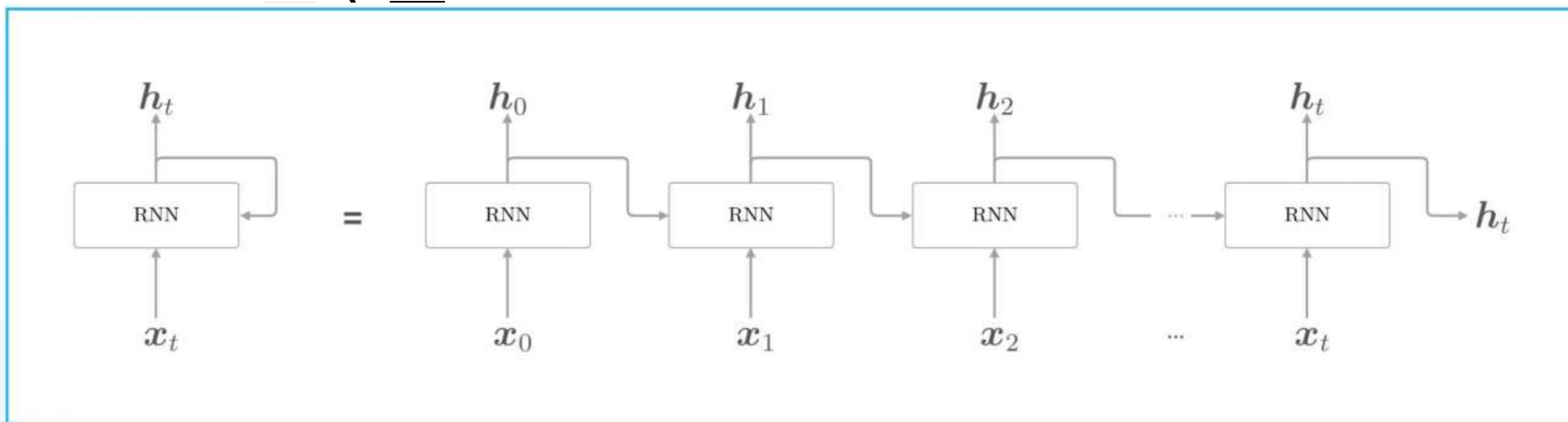
输入层



RNN存在问题

❑ RNN: 不擅长学习时序数据的长期依赖关系

❑ 梯度爆炸或梯度消散



❑ 当输入时序数据 x_t 时，RNN层输出 h_t 。这个 h_t 也称为RNN层的隐藏状态，它记录过去的信息

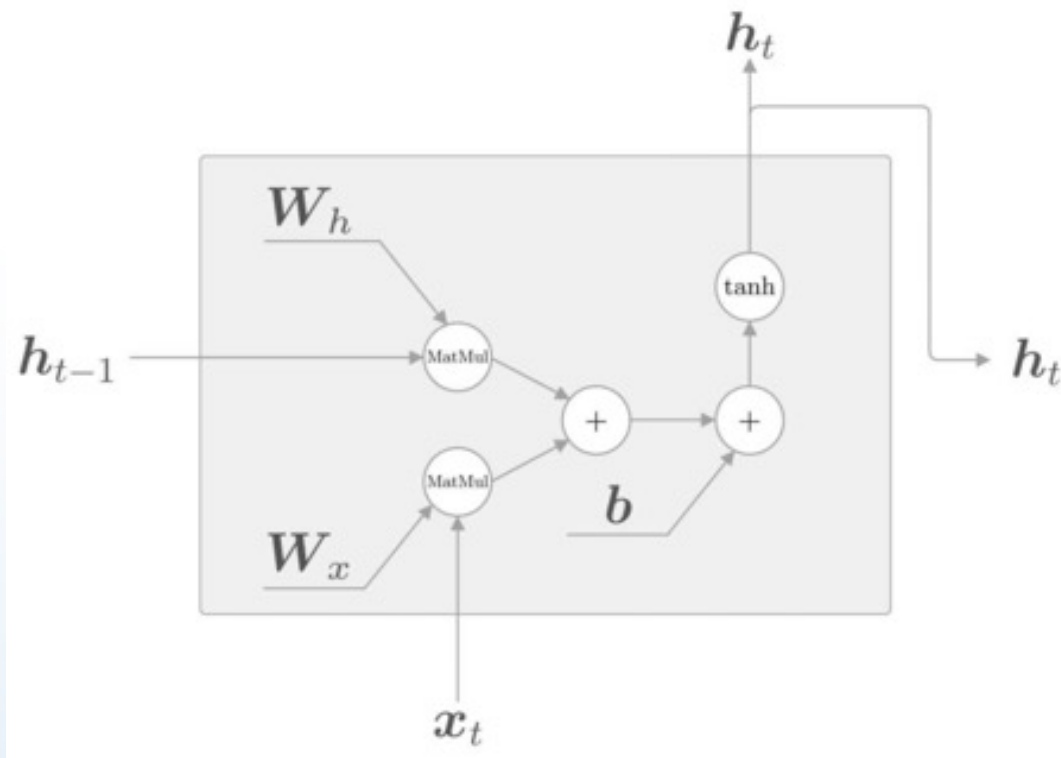
RNN存在问题

□ 用计算图来表示t时刻RNN层进行的处理

□ RNN层的正向传播

进行的计算：

- 矩阵乘积
- 矩阵加法
- 基于激活函数 tanh 的变换

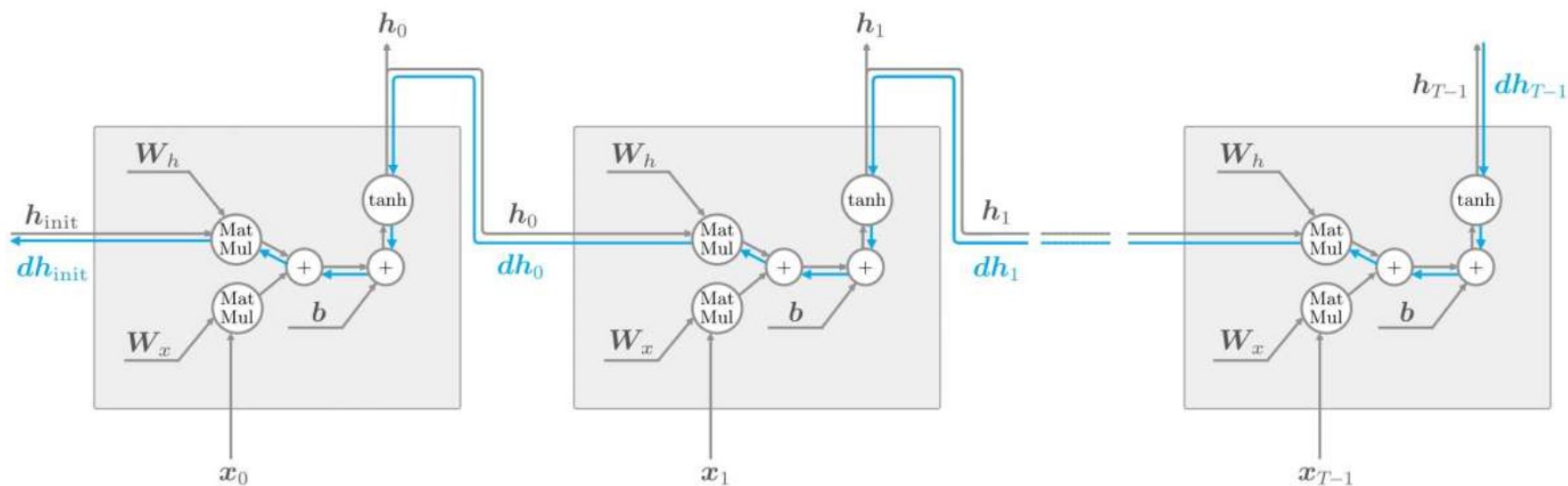


$$h_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$$

$$y = \text{Softmax}(W^{hy}h_n + b^{hy})$$

RNN存在问题

□ RNN层在时间方向上的梯度传播



□ “+”的反向传播将上游传来的梯度原样传给下游，因此梯度的值不变

RNN存在问题

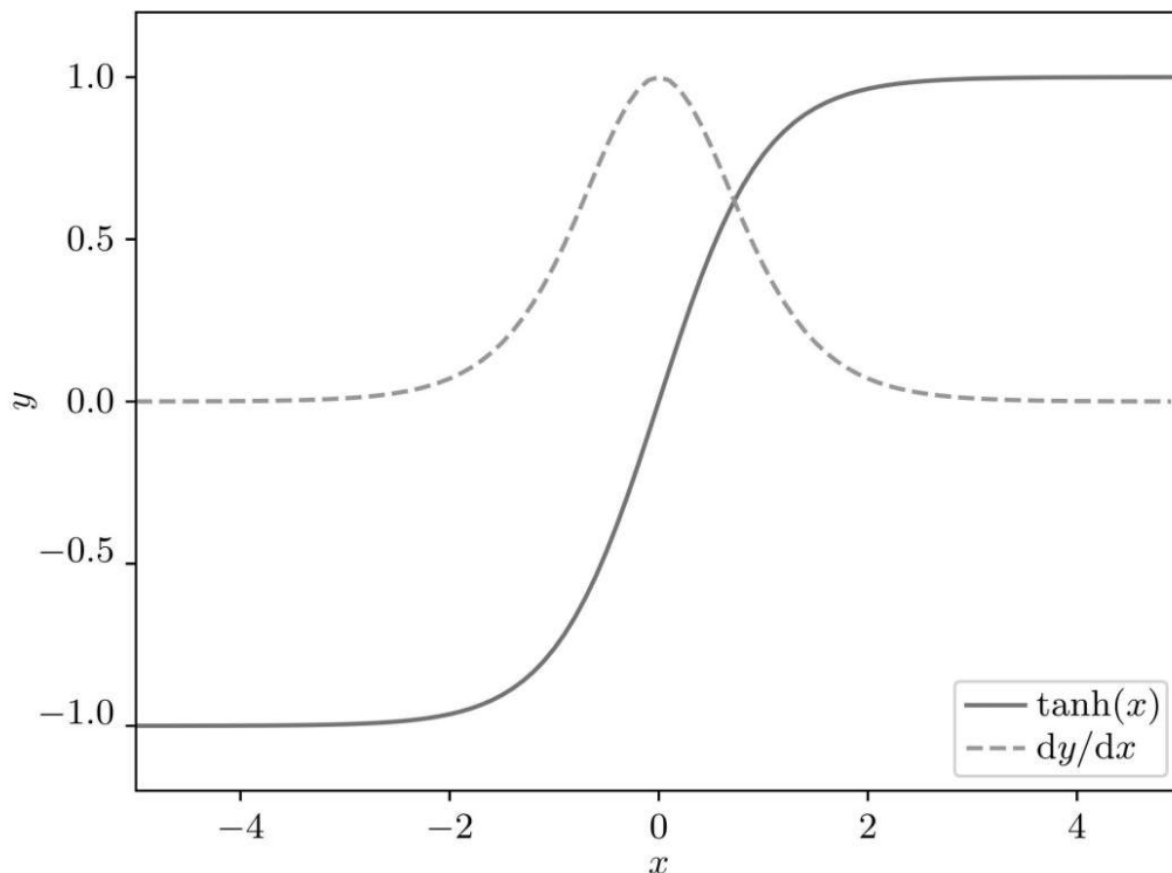
□ RNN层在时间方向上的梯度传播

□ 当 $y=\tanh(x)$ 时，其导数是 $\frac{dy}{dx} = 1 - y^2$

值小于1.0，并且随着 x 远离0，它的值在变小。

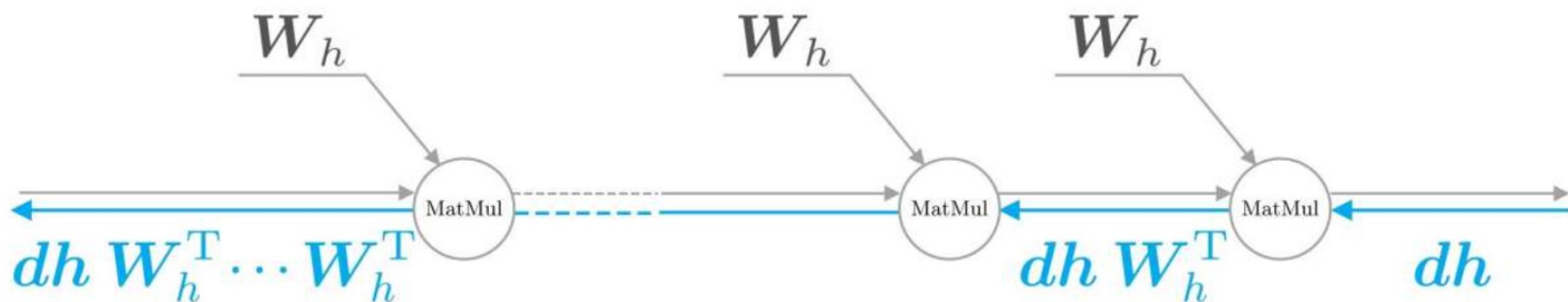
当反向传播的梯度经过tanh节点时，值会越来越小。

如果经过tanh函数 T 次，则梯度也会减小 T 次。



RNN存在问题

- ❑ RNN层在时间方向上的梯度传播
- ❑ 仅关注RNN层的矩阵乘积(MatMul)时的反向传播的梯度

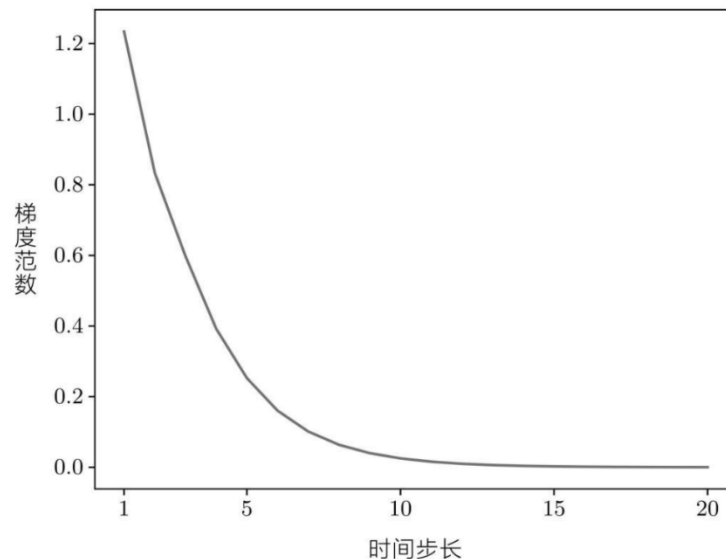
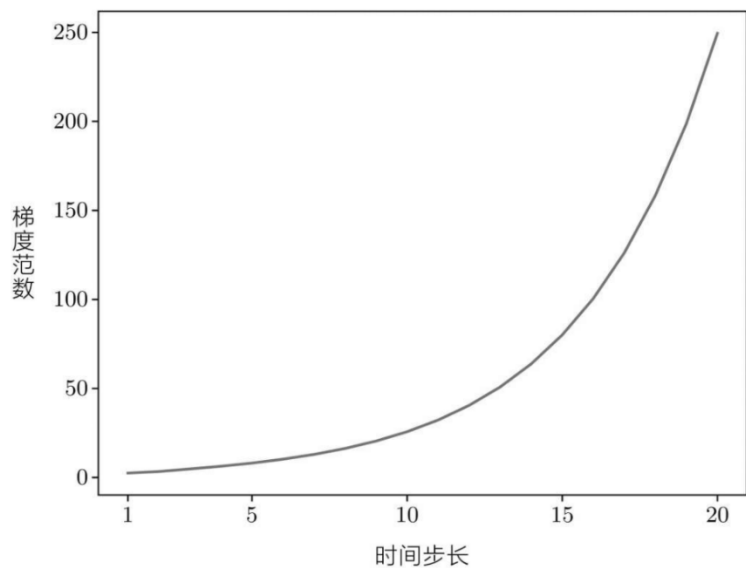


- ❑ 从上游传来梯度 dh ，此时MatMul节点的反向传播通过矩阵乘积 $dh W_h^T$ 计算梯度。
- ❑ 根据时序数据的时间步长，将这个计算重复相应次数。每一次矩阵乘积计算都使用相同的权重 W_h 。

RNN存在问题

□ RNN层在时间方向上的梯度传播

- 矩阵奇异值的最大值大于1，梯度很有可能会呈指数级增加
- 奇异值的最大值小于1，梯度会呈指数级减小



RNN存在问题

□ RNN: 不擅长学习时序数据的长期依赖关系

□ 梯度爆炸或梯度消散

□ 原因：

□ 激活函数 $\tanh(x)$

□ 使用相同的权重 W_h

□ 解决：

□ 更换激活函数

□ 梯度截断

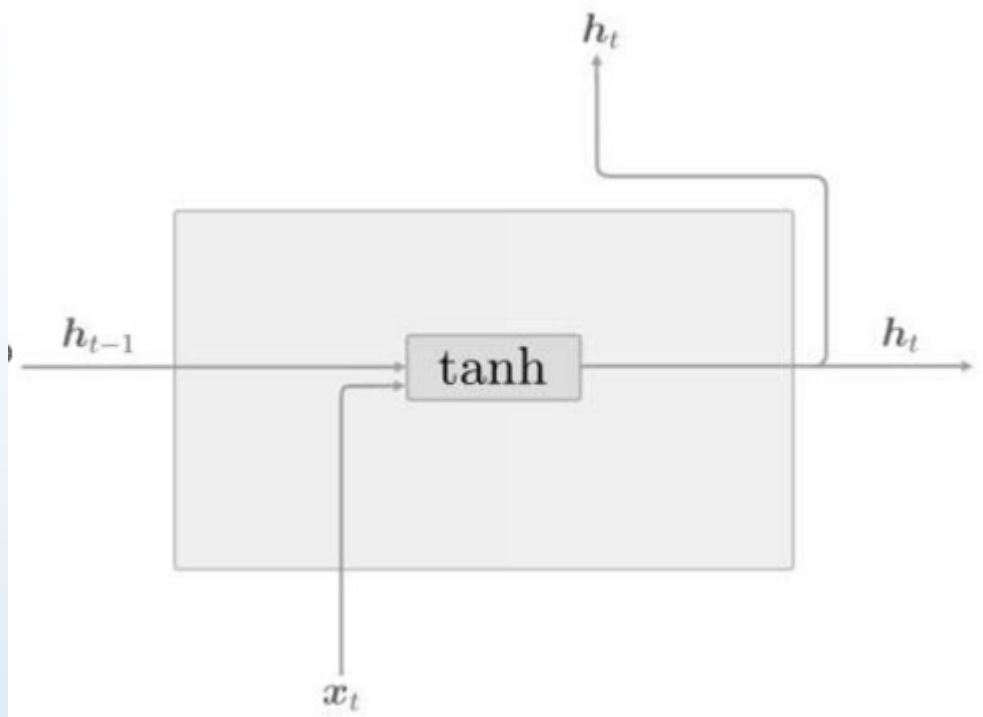
if $\|\hat{g}\| \geq threshold$:

$$\hat{g} = \frac{threshold}{\|\hat{g}\|} \hat{g}$$

循环神经网络-长短时记忆网络 (LSTM)

□ LSTM的接口 $h_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$

□ RNN简略图示法



循环神经网络-长短时记忆网络 (LSTM)

□ RNN层与LSTM层的比较

□ LSTM的记忆单元 c ：LSTM专用记忆部门

- 记忆单元仅在LSTM层内部接收和传递数据

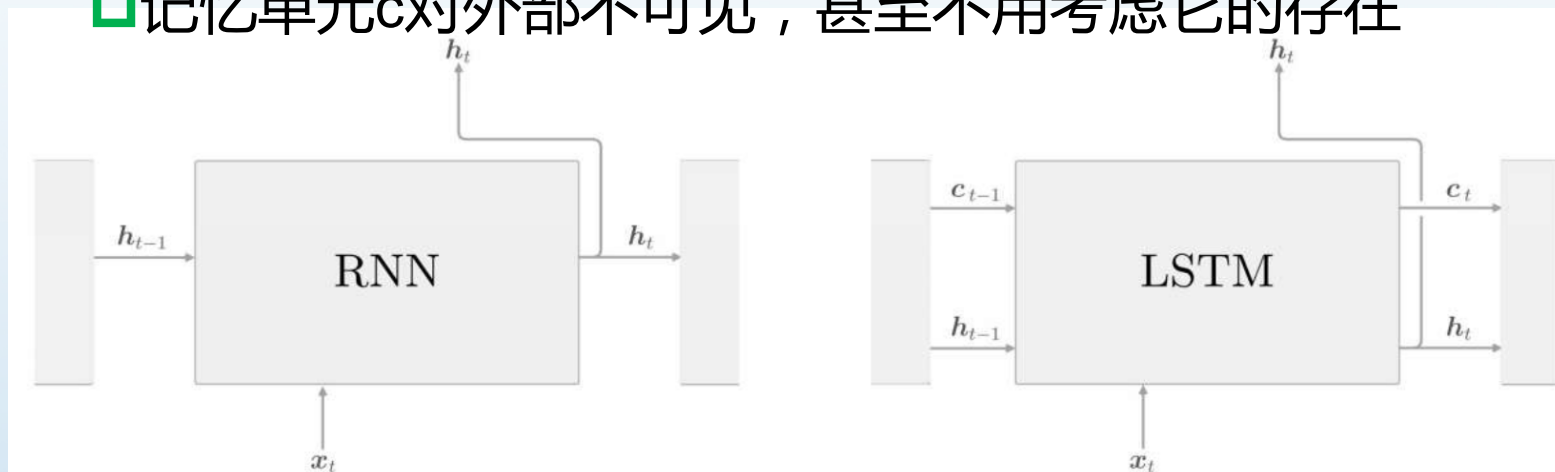
- 在LSTM层内部结束工作，不向其他层输出

□ LSTM的隐藏状态 h ：与RNN层相同，被输出到其他层

□ LSTM的输出侧：

- 仅有隐藏状态向量 h

- 记忆单元 c 对外部不可见，甚至不用考虑它的存在

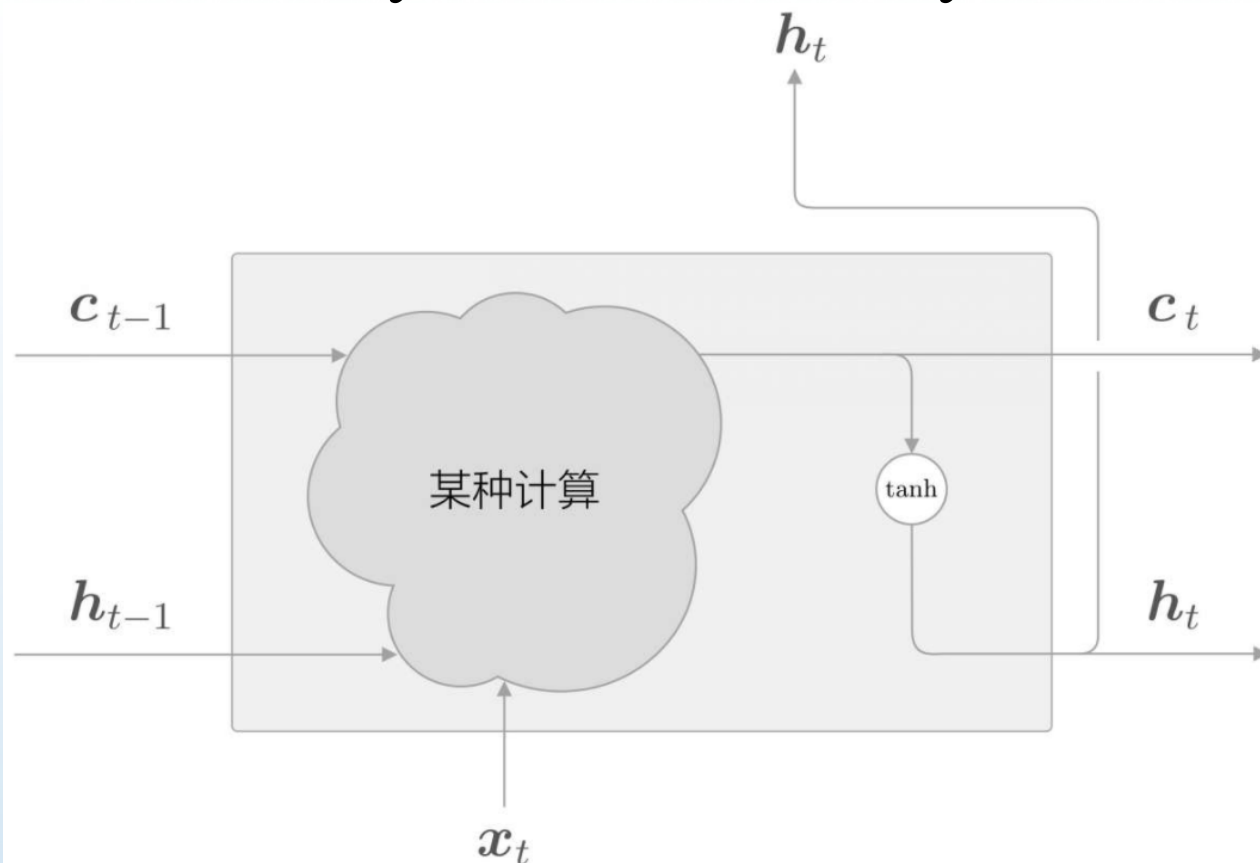


LSTM层的结构

□ 记忆单元 C_t ：存储了时刻 t 时LSTM的记忆

□ C_t 保存了从过去到时刻 t 的所有必要信息

□ 基于 C_t ，计算隐藏状态 h_t



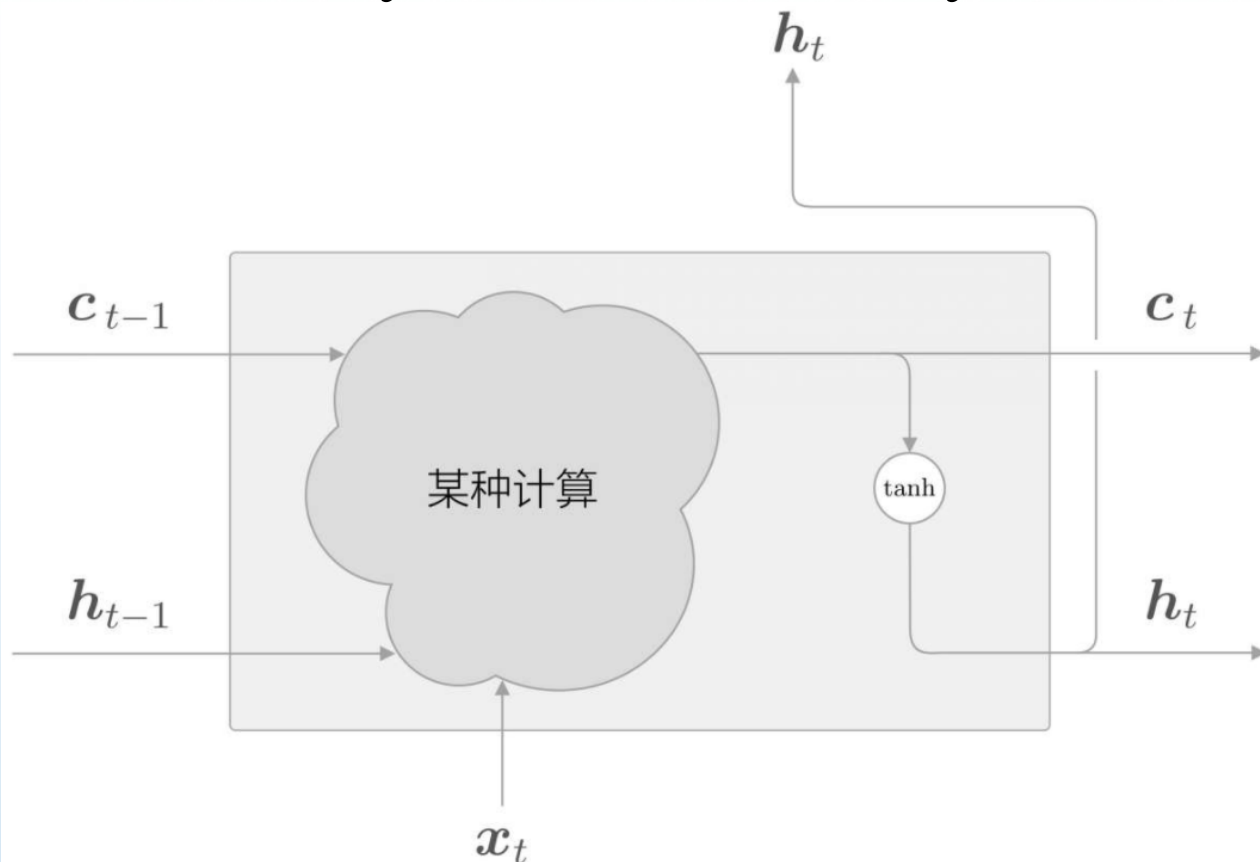
□ C_t ：基于3个输入 C_{t-1} 、 h_{t-1} 和 x_t ，经过“某种计算”算出来

LSTM层的结构

□ 记忆单元 c_t ：存储了时刻 t 时LSTM的记忆

□ c_t 保存了从过去到时刻 t 的所有必要信息

□ 基于 c_t ，计算隐藏状态 h_t

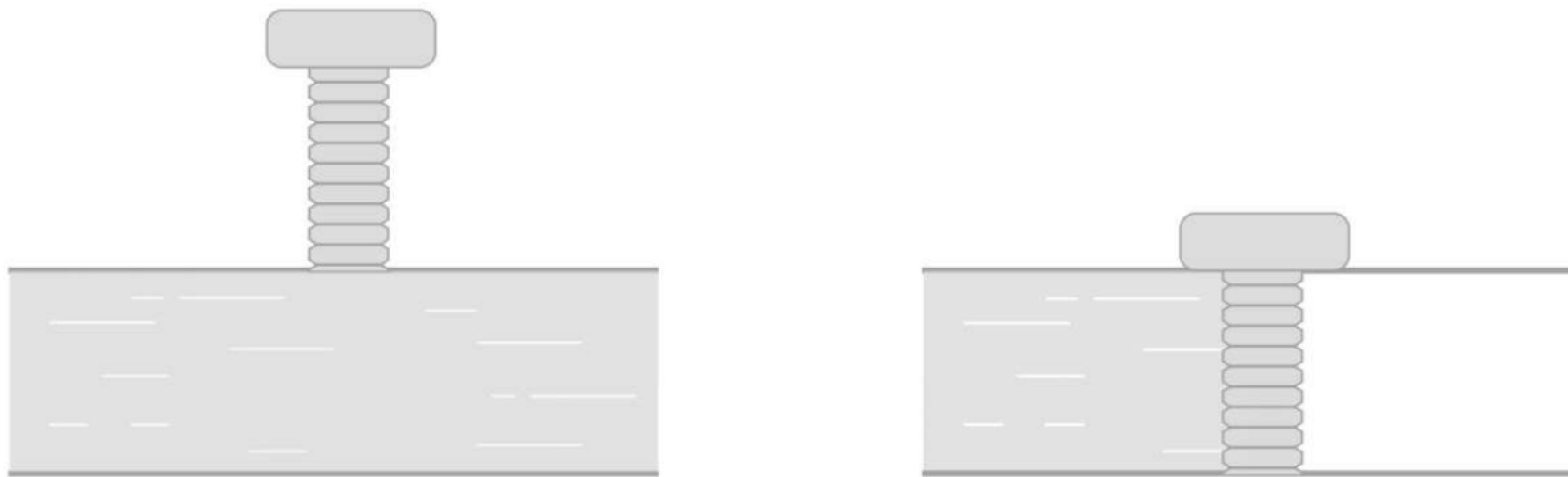


□ $h_t = \tanh(c_t)$,
表示对的 h_t 各个元素应用
tanh函数

□ h_t 和 c_t 元素个数相同

LSTM层的结构

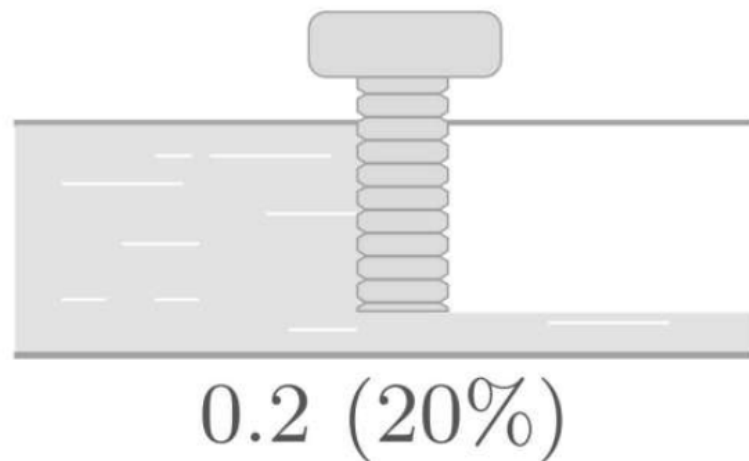
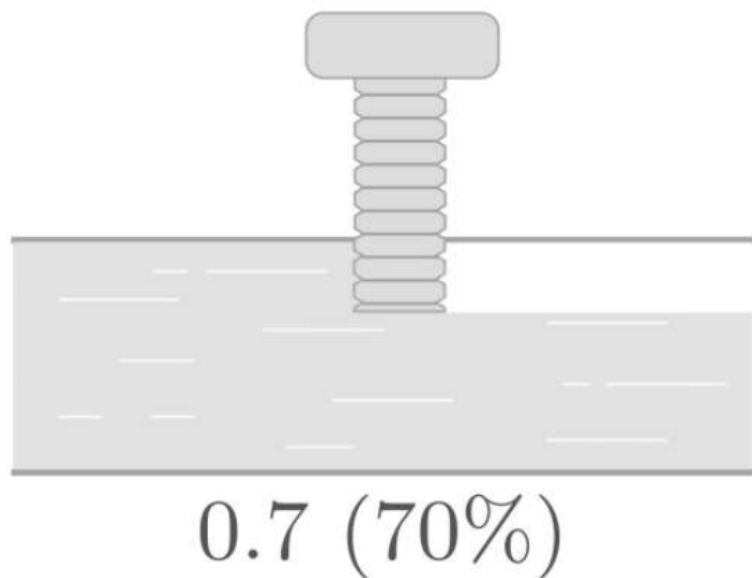
□ Gate的功能：门打开或合上一样，控制数据的流动



□ LSTM中使用的门并非只能“开或合”，还可以根据将门打开多少来控制水的流量

LSTM层的结构

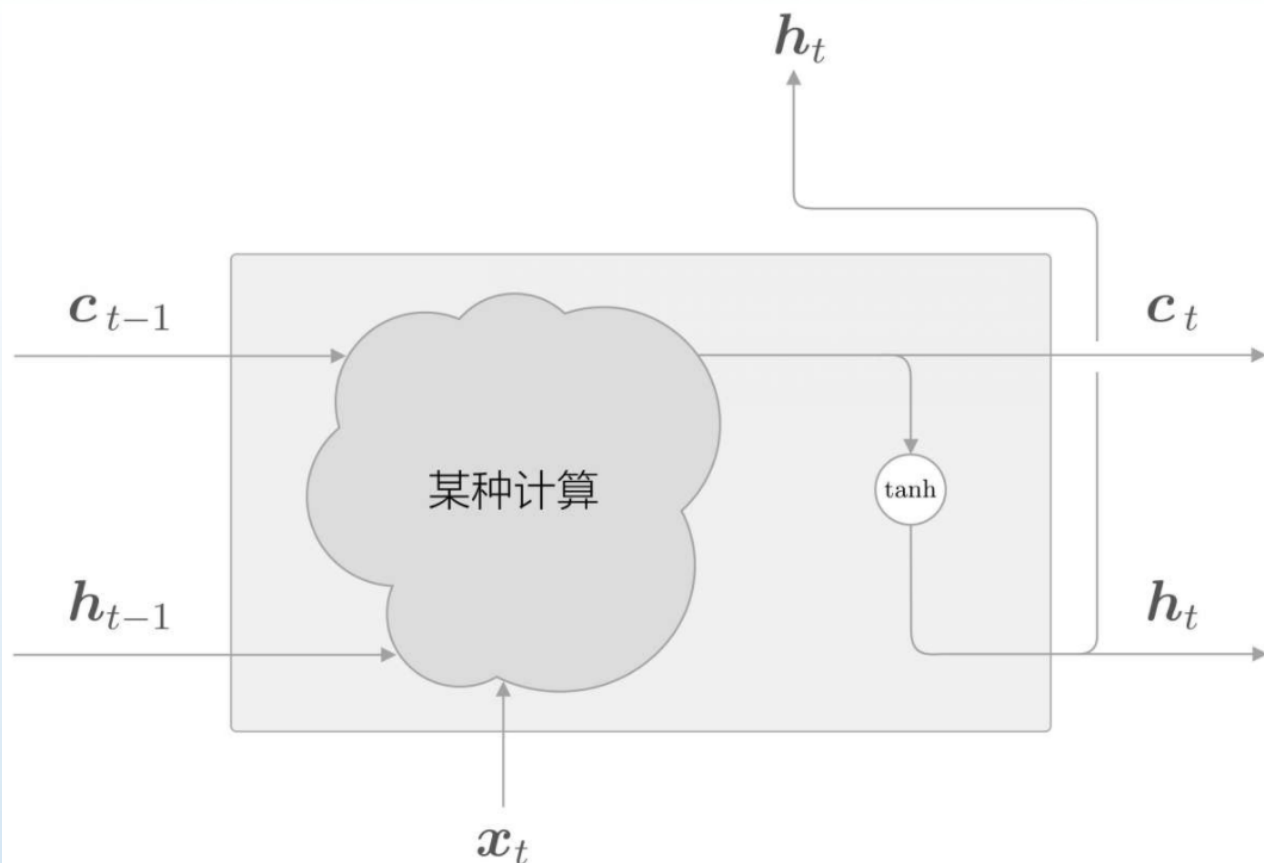
- 将“开合程度”控制在0.7 (70%) 或者0.2 (20%)



- 门的开合程度由0.0 ~ 1.0的实数表示(1.0为全开)，通过这个数值控制流出的水量
- 有专门的权重参数用于控制门的开合程度，这些权重参数通过学习被更新。sigmoid函数用于求门的开合程度

LSTM层的结构

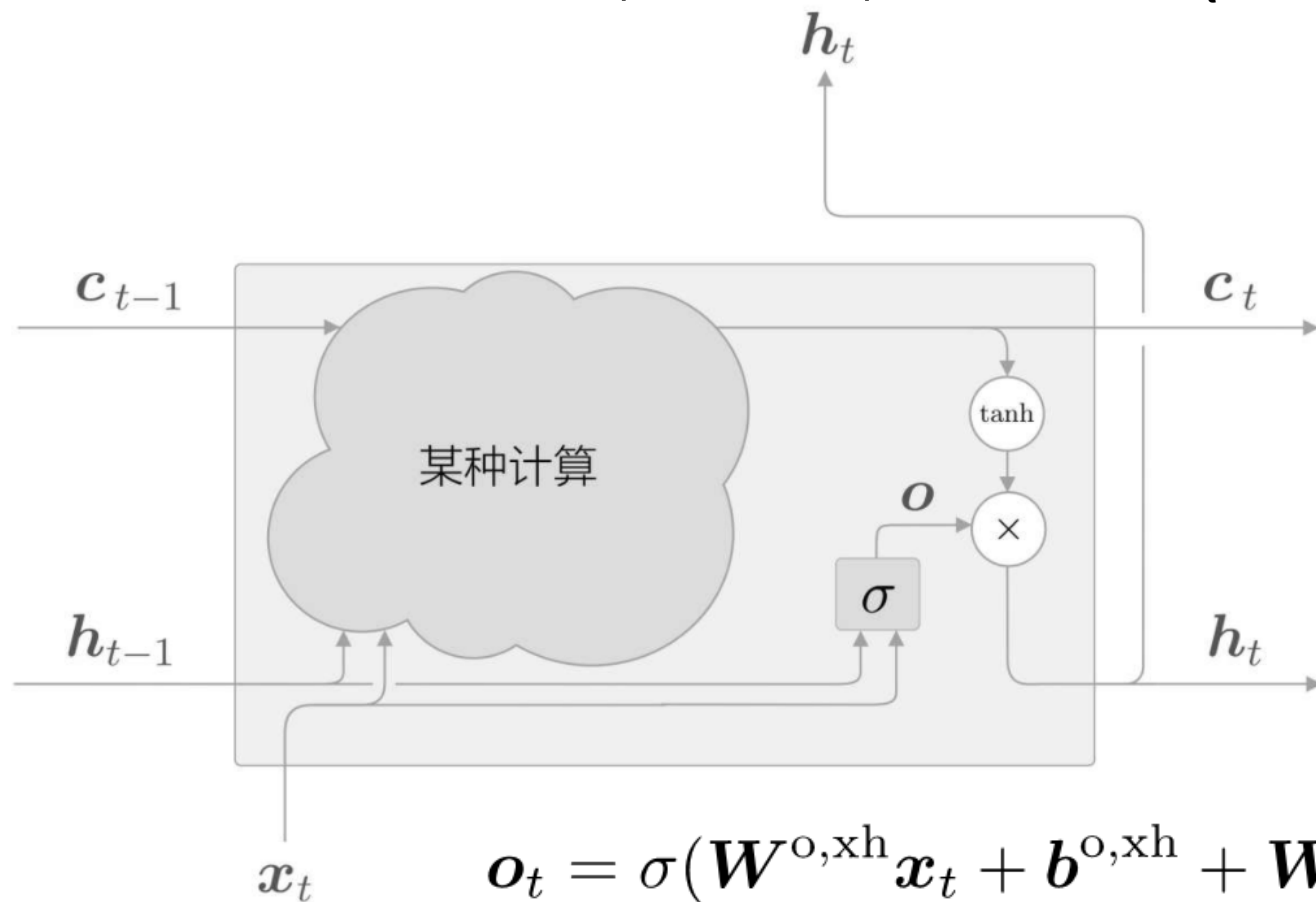
- ❑ 隐藏状态 h_t ：对记忆单元 c_t 仅仅应用了tanh函数
- ❑ 对 $\tanh(c_t)$ 施加门：调整它们作为下一时刻的隐藏状态的重要程度
- ❑ 输出门(output gate)：门管理下一个隐藏状态 h_t 的输出



LSTM层的结构

□ 添加输出门

□ 输出门的开合程度(流出比例)：根据输入 x_t 和上一个状态 h_{t-1} 求出



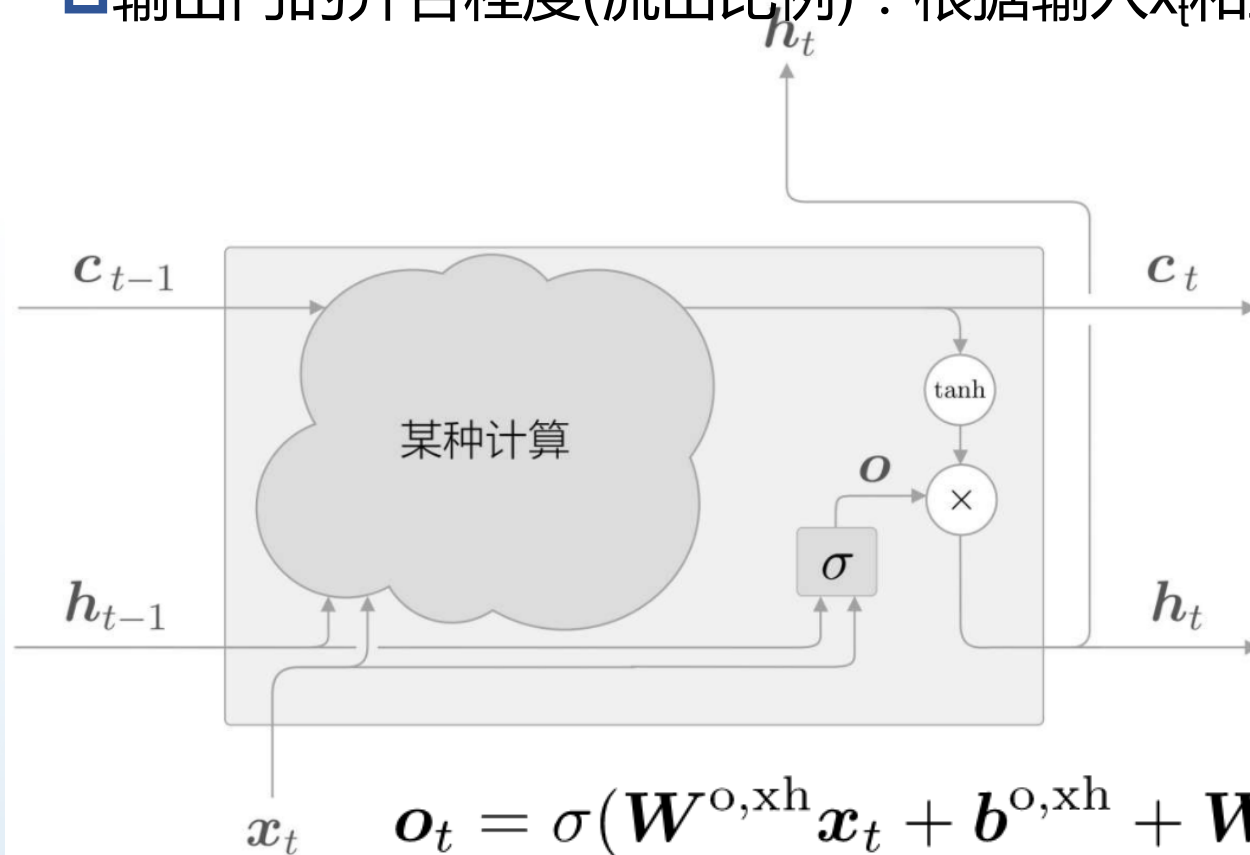
$$o_t = \sigma(W^{o,xh}x_t + b^{o,xh} + W^{o,hh}h_{t-1} + b^{o,hh})$$

$$h_t = o_t \odot \tanh(c_t)$$

LSTM层的结构

□ 添加输出门

□ 输出门的开合程度(流出比例)：根据输入 x_t 和上一个状态 h_{t-1} 求出



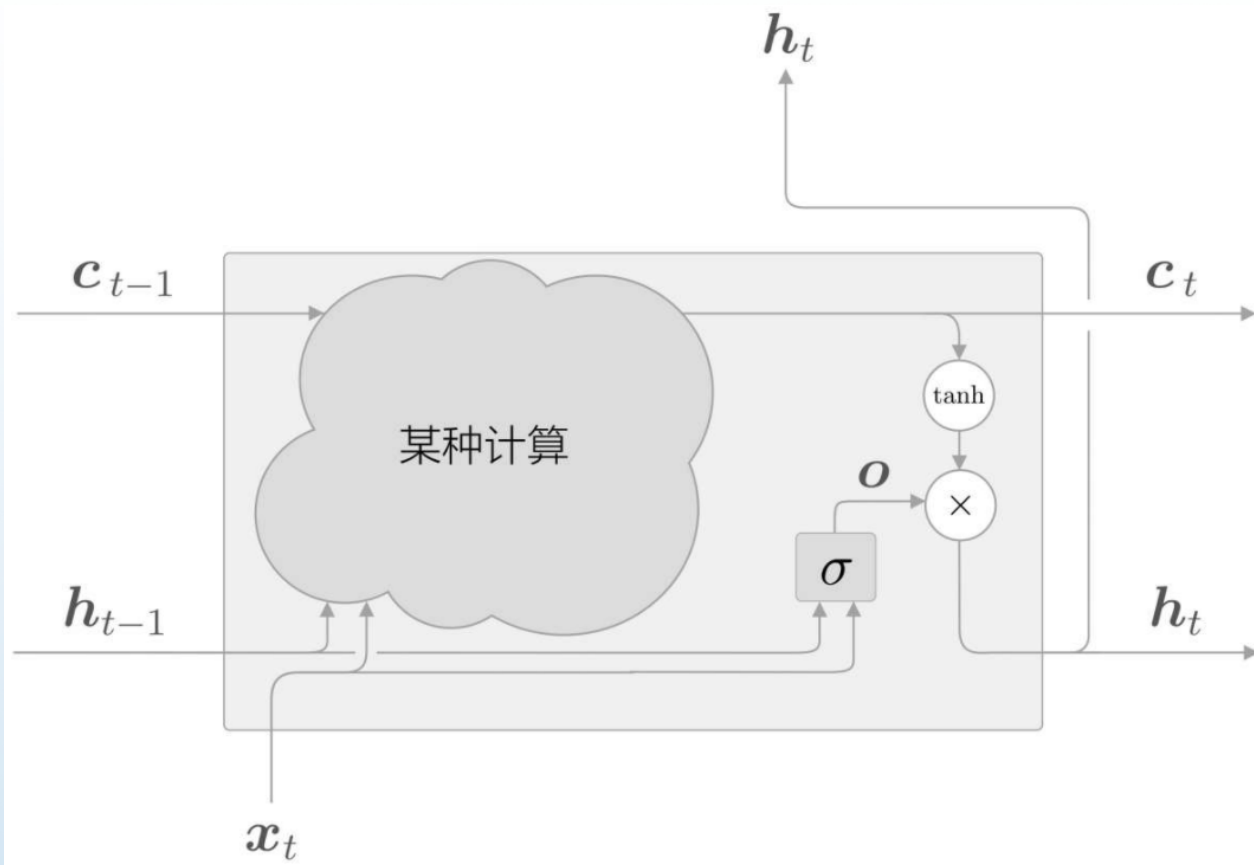
$$o_t = \sigma(W^{o,xh}x_t + b^{o,xh} + W^{o,hh}h_{t-1} + b^{o,hh})$$

$$h_t = o_t \odot \tanh(c_t)$$

$\odot$ Hardamard乘积

LSTM层的结构

函数	范围	应用
tanh函数	实数(-1.0 ~ 1.0)	包含实质信息的数据，表示某种被编码的“信息”的强弱（程度）
sigmoid函数	实数（0.0 ~ 1.0）	门：表示数据流出的比例

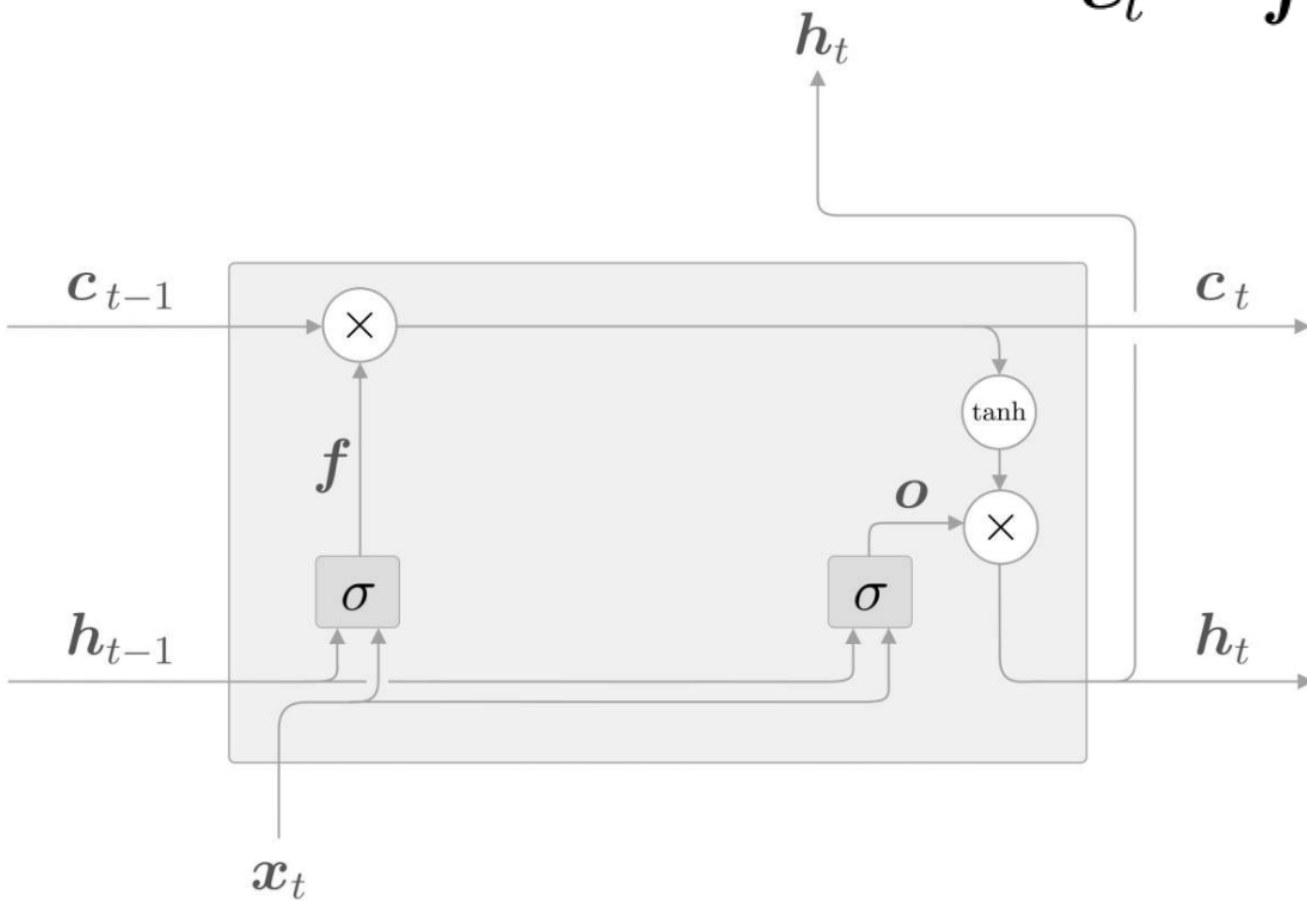


LSTM层的结构

❑遗忘门: 记忆单元 c_{t-1} 上添加一个忘记不必要记忆的门

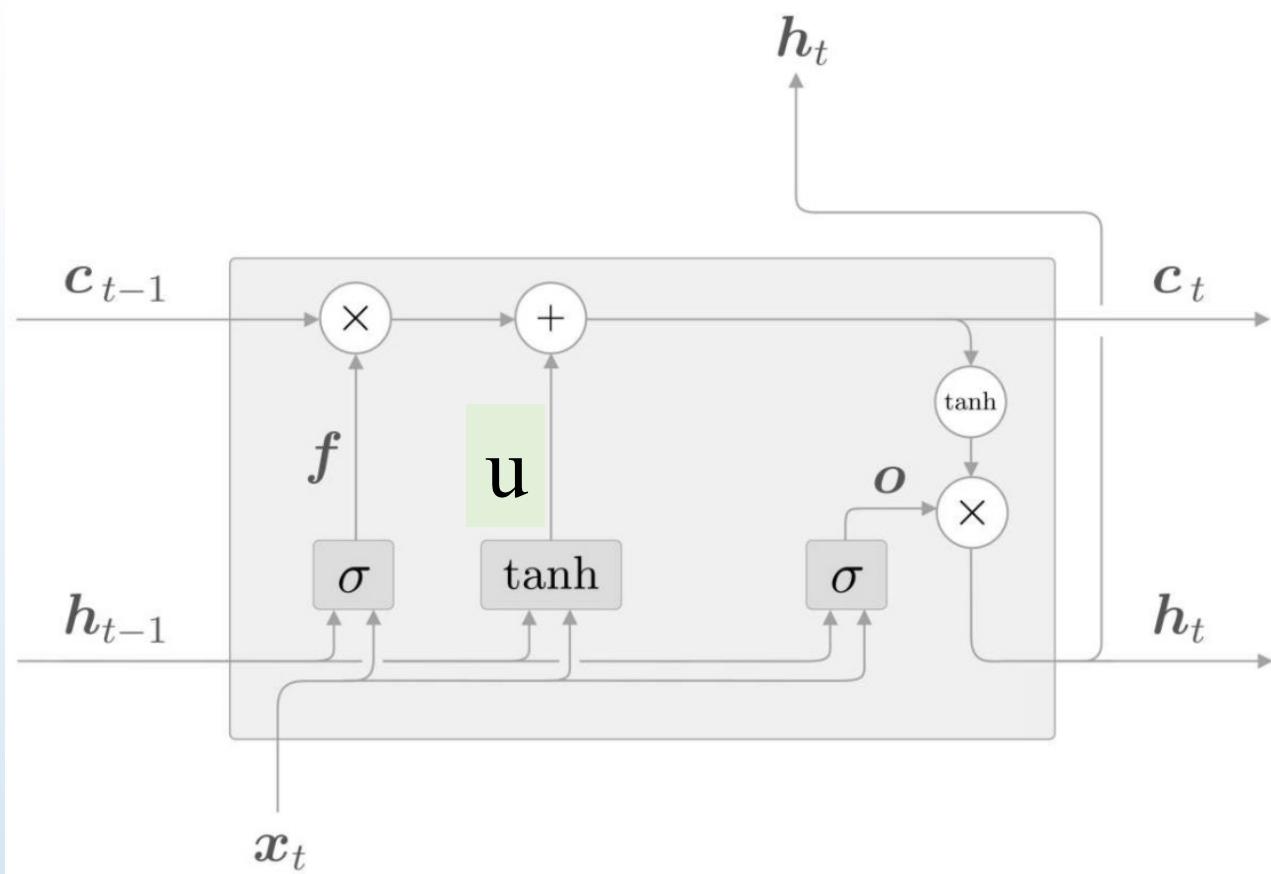
$$f_t = \sigma(\mathbf{W}^{f,xh} \mathbf{x}_t + \mathbf{b}^{f,xh} + \mathbf{W}^{f,hh} \mathbf{h}_{t-1} + \mathbf{b}^{f,hh})$$

$$\mathbf{c}_t = f_t \odot \mathbf{c}_{t-1}$$



LSTM层的结构

- ❑遗忘门：从上一时刻的记忆单元中删除了应该忘记的东西
- ❑新的记忆单元：向新的记忆单元添加必要信息
- ❑添加新的tanh节点

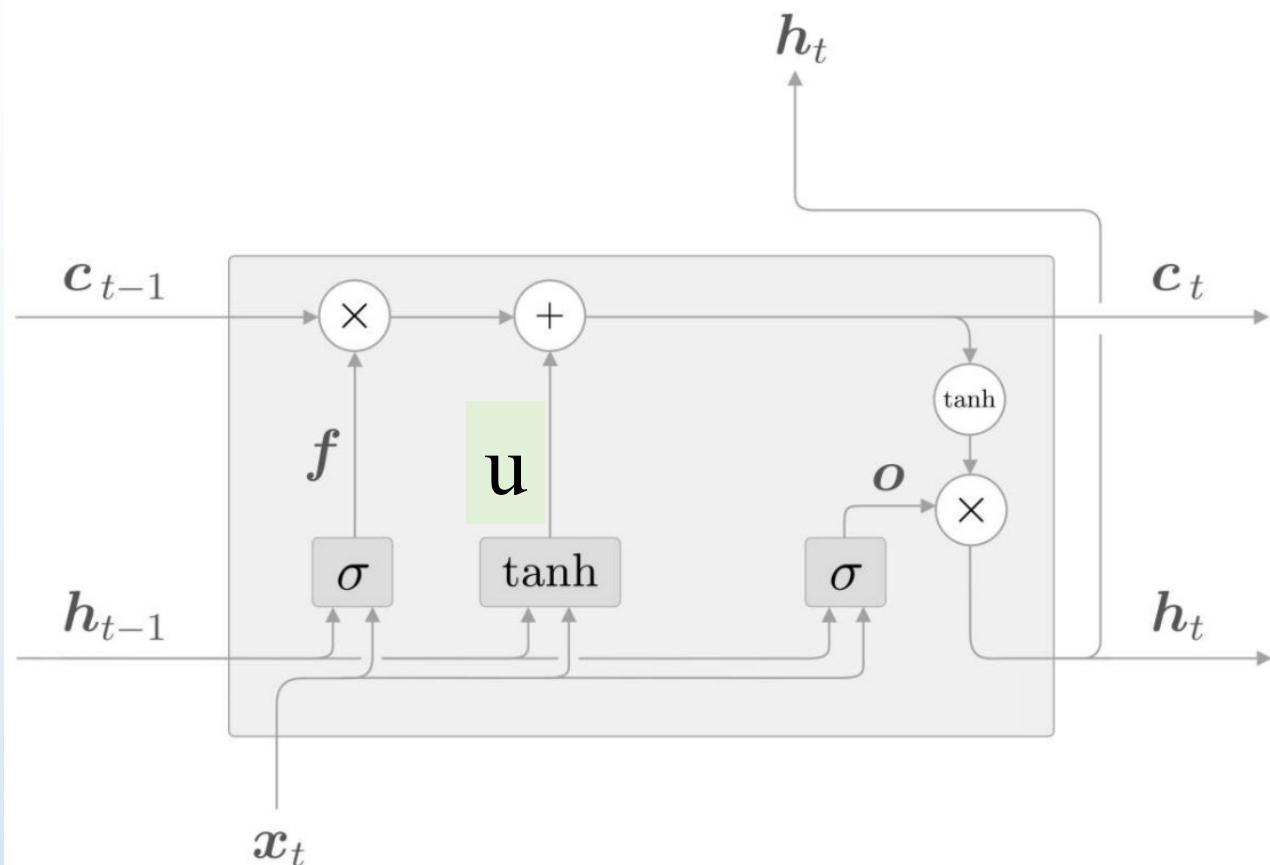


LSTM层的结构

□ 添加新的tanh节点u：表示向记忆单元添加的新信息

□ 将u加到上一时刻的 c_{t-1} 上，从而形成新的记忆

$$u_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$$



LSTM层的结构

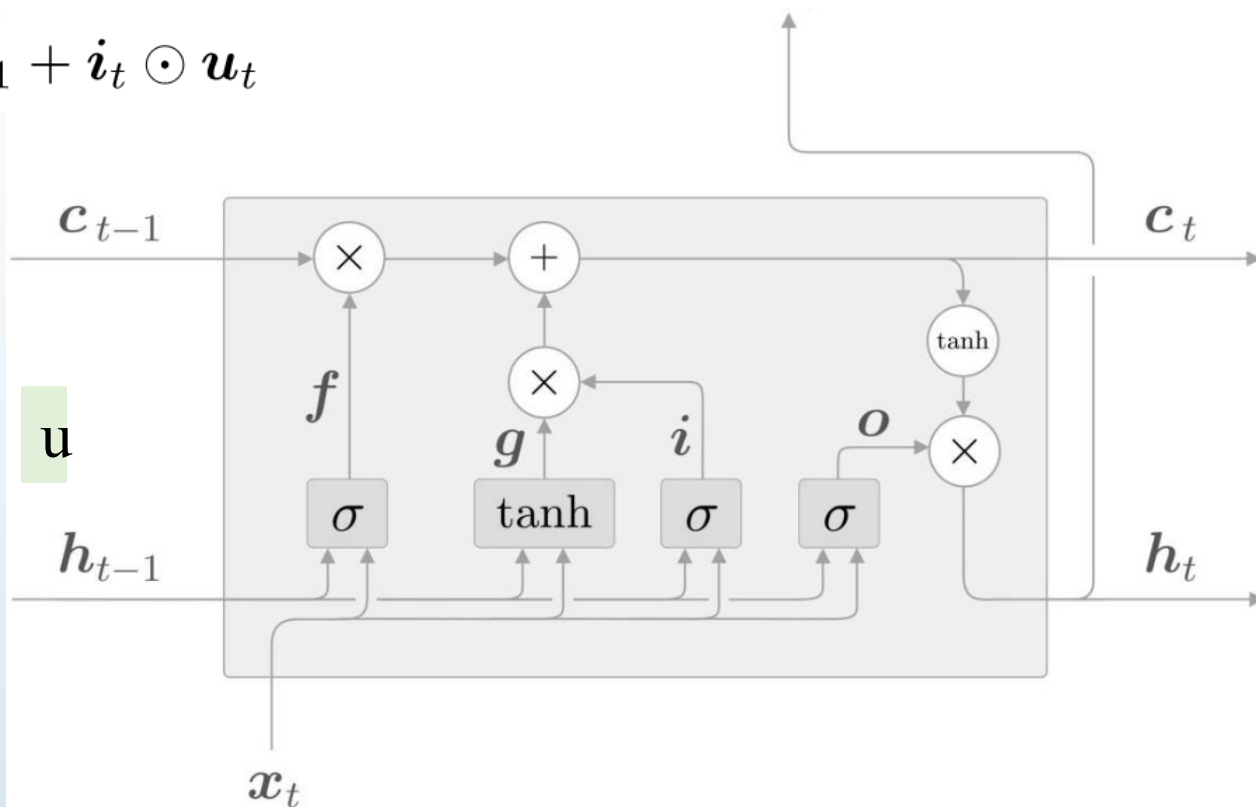
□输入门(input gate):为u添加门

□判断新增信息u的各个元素的价值有多大

□对新添加新信息的加权后再添加

$$i_t = \sigma(\mathbf{W}^{i,xh} \mathbf{x}_t + \mathbf{b}^{i,xh} + \mathbf{W}^{i,hh} \mathbf{h}_{t-1} + \mathbf{b}^{i,hh})$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{u}_t$$



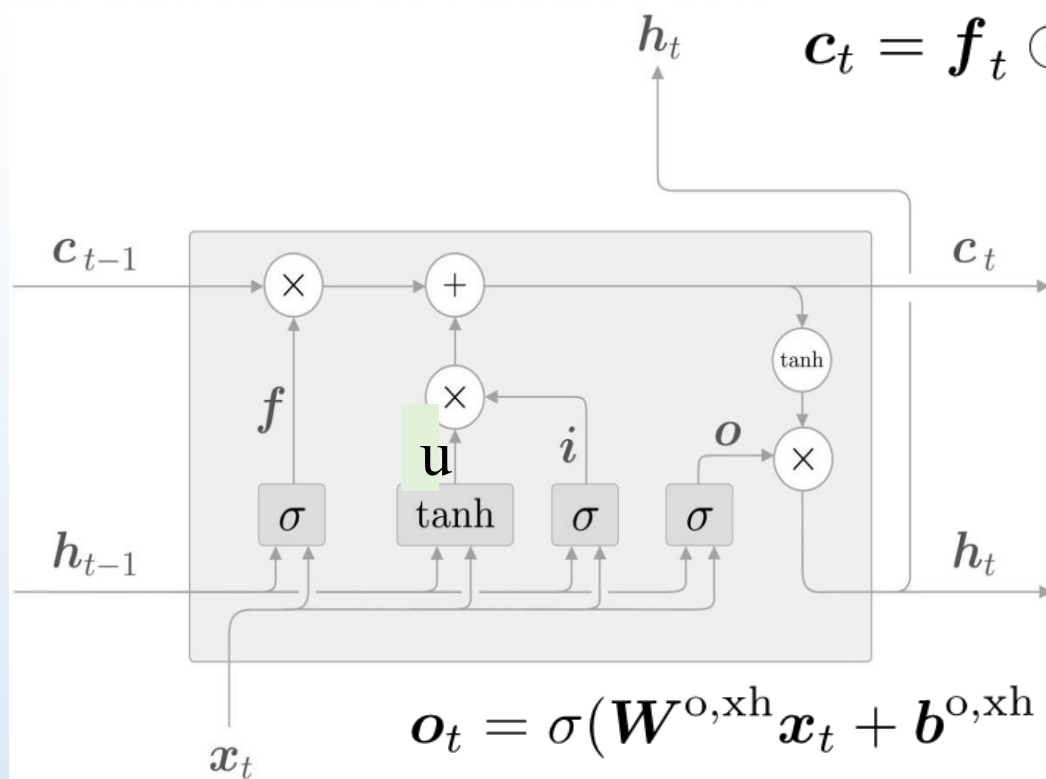
LSTM层的结构

$$\mathbf{u}_t = \tanh(\mathbf{W}^{\text{xh}} \mathbf{x}_t + \mathbf{b}^{\text{xh}} + \mathbf{W}^{\text{hh}} \mathbf{h}_{t-1} + \mathbf{b}^{\text{hh}})$$

$$\mathbf{i}_t = \sigma(\mathbf{W}^{\text{i,xh}} \mathbf{x}_t + \mathbf{b}^{\text{i,xh}} + \mathbf{W}^{\text{i,hh}} \mathbf{h}_{t-1} + \mathbf{b}^{\text{i,hh}})$$

$$\mathbf{f}_t = \sigma(\mathbf{W}^{\text{f,xh}} \mathbf{x}_t + \mathbf{b}^{\text{f,xh}} + \mathbf{W}^{\text{f,hh}} \mathbf{h}_{t-1} + \mathbf{b}^{\text{f,hh}})$$

$$\mathbf{h}_t \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{u}_t$$



$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}^{\text{o,xh}} \mathbf{x}_t + \mathbf{b}^{\text{o,xh}} + \mathbf{W}^{\text{o,hh}} \mathbf{h}_{t-1} + \mathbf{b}^{\text{o,hh}})$$

□ LSTM的梯度的流动

□ 记忆单元的反向传播

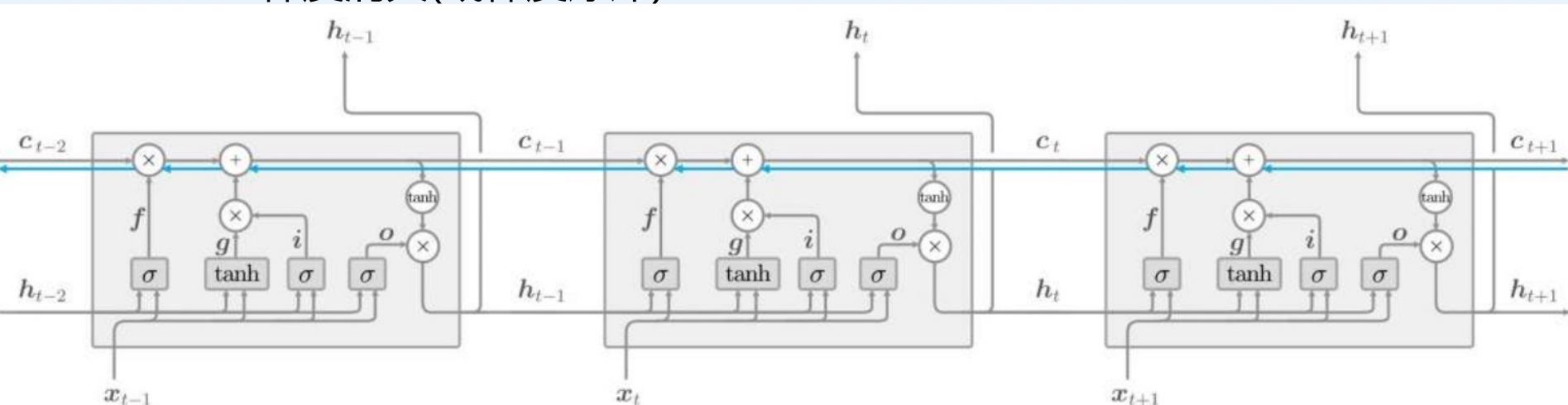
□ 反向传播仅流过 “+” 和 “×” 节点

- “+” 节点：将上游传来的梯度原样流出，梯度无变化(退化)

- “×” 节点：计算的是对应元素的乘积，且每次都会基于不同的门值进行对应元素的乘积计算

- 不会发生梯度消失(或梯度爆炸)的原因

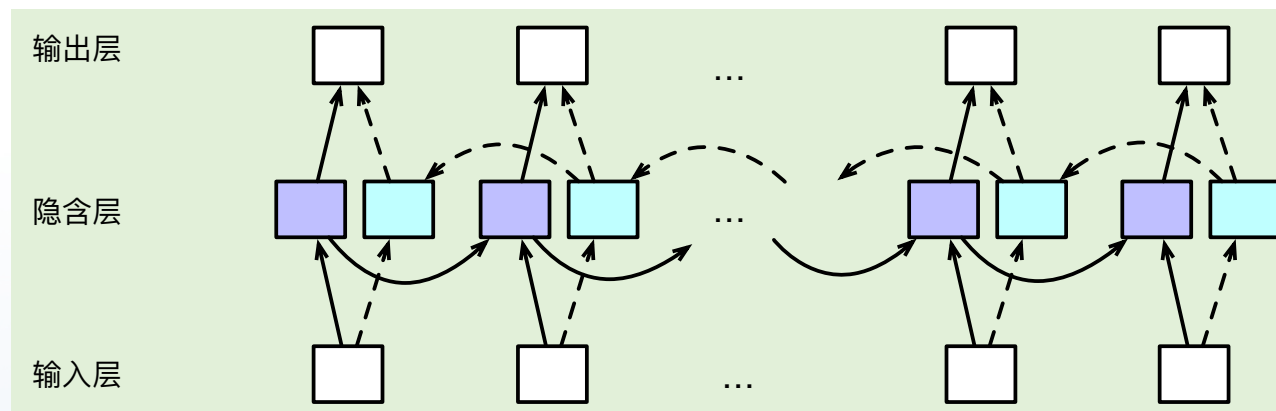
- RNN反向传播：使用相同的权重矩阵重复了多次矩阵乘积计算，导致了梯度消失(或梯度爆炸)



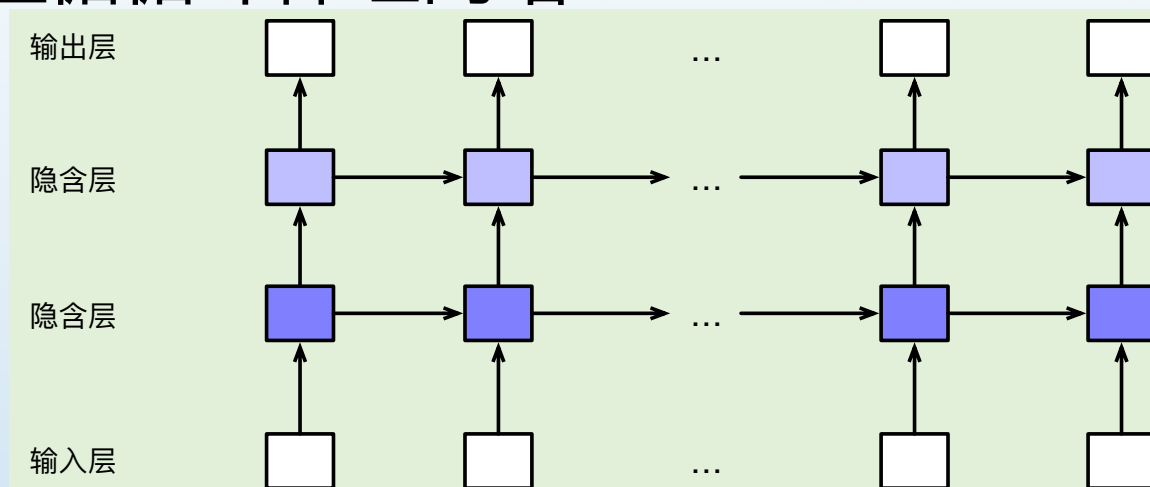
循环神经网络的改进

□ 双向循环神经网络

□ 词性标注：词性与前面和后面的词有关



□ 堆叠循环神经网络



RNN模型实现

- ❑ RNN类: RNN在PyTorch的torch.nn包中有相应的实现
- ❑ 构造函数: 至少需要提供两个参数
 - ❑ input_size: 每个时刻输入的大小
 - ❑ hidden_size: 表示隐含层的大小
 - ❑ batch_first: 设为True(其默认值为False), 输入和输出的第1维代表批次的大小(即一次同时处理序列的数目)
- ❑ 调用该RNN对象
 - ❑ 输入数据: 形状为 (batch , seq_len , input_size

RNN模型实现

```
>>> from torch.nn import RNN
>>> rnn = RNN(input_size=4, hidden_size=5, batch_first=True)
    # 定义一个RNN, 每个时刻输入大小为4, 隐含层大小为5
>>> inputs = torch.rand(2, 3, 4) # 输入数据批次大小为2, 即有两个序列, 每个序列的
    # 长度为3, 每个时刻输入大小为4
>>> outputs, hn = rnn(inputs)
    # outputs为输出序列的隐含层, hn为最后一个时刻的隐含层
>>> print(outputs) # 输出两个序列, 每个序列长度为3, 大小为5
tensor([[[[-0.3370,  0.1573,  0.1213, -0.0054, -0.1670],
          [-0.4587, -0.0574,  0.0305,  0.2515, -0.1272],
          [-0.5635, -0.0570,  0.1677,  0.3289, -0.1813]],

        [[[-0.3184,  0.1510,  0.0625,  0.0258, -0.2403],
          [-0.5192, -0.2856,  0.0590,  0.3002, -0.0541],
          [-0.3684, -0.1418,  0.5262,  0.1038, -0.1735]]],
        grad_fn=<TransposeBackward1>)
>>> print(hn) # 最后一个时刻的隐含层, 值与outputs中最后一个时刻相同
tensor([[[[-0.5635, -0.0570,  0.1677,  0.3289, -0.1813],
          [-0.3684, -0.1418,  0.5262,  0.1038, -0.1735]]],
        grad_fn=<StackBackward>)
>>> print(outputs.shape, hn.shape) # 输出隐含层序列和最后一个时刻隐含层的形状,
    # 分别为(2, 3, 5), 即批次大小、序列长度和隐含层大小, 以及(1, 2, 5),
    # 即1、批次大小和隐含层大小
torch.Size([2, 3, 5]) torch.Size([1, 2, 5])
```


LSTM模型实现

- ❑ LSTM类：torch.nn包中提供
 - ❑ 初始化的参数以及输入数据与RNN相同
 - ❑ 输出数据：
 - ❑ H_n ：最后一个时刻的隐含层
 - ❑ C_n ：最后一个时刻的记忆细胞

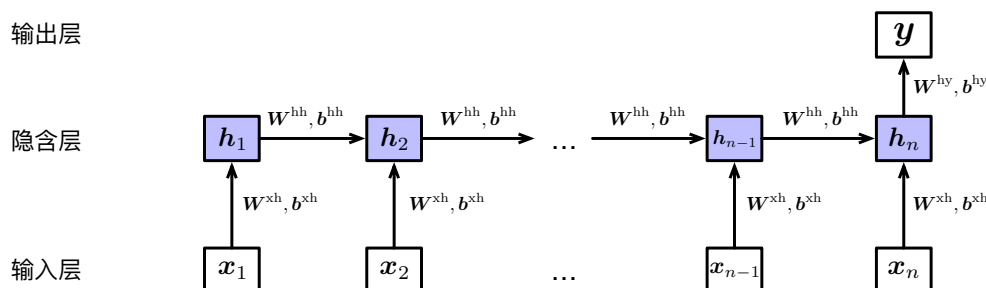
LSTM模型实现

```
>>> from torch.nn import LSTM
>>> lstm = LSTM(input_size=4, hidden_size=5, batch_first=True)
>>> inputs = torch.rand(2, 3, 4)
>>> outputs, (hn, cn) = lstm(inputs) # outputs为输出序列的隐含层, hn为最后一个
# 时刻的隐含层, cn为最后一个时刻的记忆细胞
>>> print(outputs) # 输出两个序列, 每个序列长度为3, 大小为5
tensor([[[[-0.1921, -0.0125,  0.0018,  0.0676,  0.0157],
          [-0.2464, -0.0565, -0.1037,  0.0957,  0.0048],
          [-0.2961, -0.0872, -0.1543,  0.1562, -0.0065]],

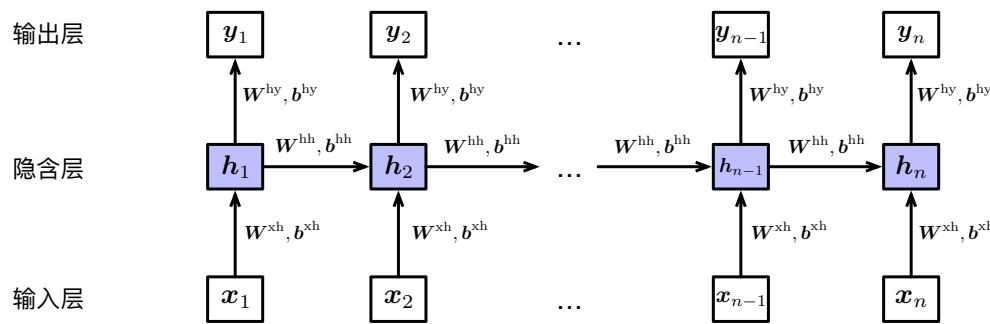
        [[[-0.2115, -0.0578, -0.0784,  0.0920,  0.0025],
          [-0.2648, -0.0526, -0.0938,  0.1610, -0.0093],
          [-0.3186, -0.0483, -0.0977,  0.2401, -0.0310]]],
        grad_fn=<TransposeBackward0>)
>>> print(hn) # 最后一个时刻的隐含层, 值与outputs中最后一个时刻相同
tensor([[[[-0.2961, -0.0872, -0.1543,  0.1562, -0.0065],
          [-0.3186, -0.0483, -0.0977,  0.2401, -0.0310]]],
        grad_fn=<StackBackward>)
>>> print(cn) # 最后一个时刻的记忆细胞
tensor([[[[-0.8748, -0.2550, -0.2490,  0.3584, -0.0286],
          [-0.8544, -0.1128, -0.1598,  0.5203, -0.1183]]],
        grad_fn=<StackBackward>)
>>> print(outputs.shape, hn.shape, cn.shape)
# 输出隐含层序列和最后一个时刻隐含层以及记忆细胞的形状
torch.Size([2, 3, 5]) torch.Size([1, 2, 5]) torch.Size([1, 2, 5])
```

循环神经网络的应用

□ 分类问题



□ 序列标注问题



□ 序列到序列问题的理解和生成问题

□ 编码器--解码器模型

□ 序列到序列模型指的是先对一个序列（如一个自然语言句子）编码，再对其解码，即生成一个新的序列

□ 机器翻译等多种应用

循环神经网络的应用

序列到序列模型

编码器：使用RNN对源语言句子编码

解码器：另一个RNN

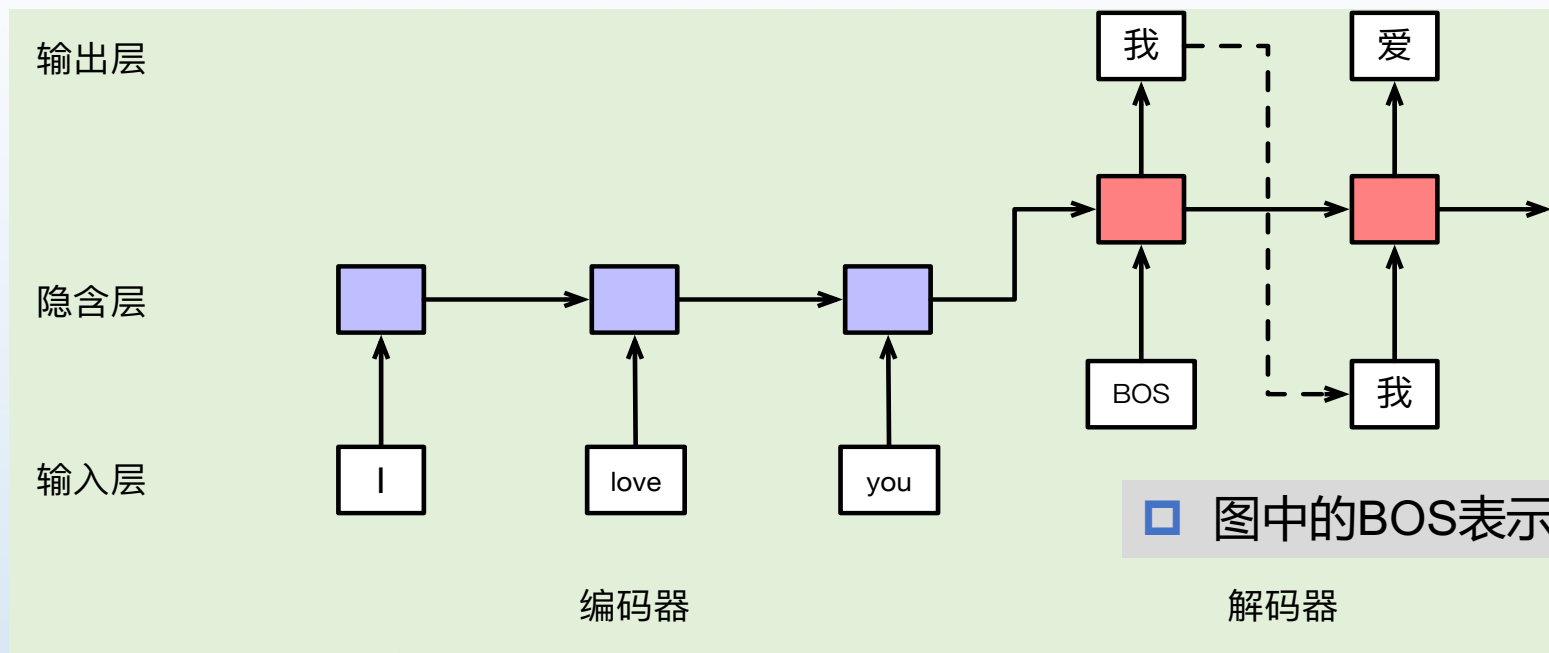
最后一个单词对应的隐含层作为初始

调用解码器逐词生成目标语言的句子

问题：基本假设原始序列的最后一个隐含状态包含了该序列的全部信息

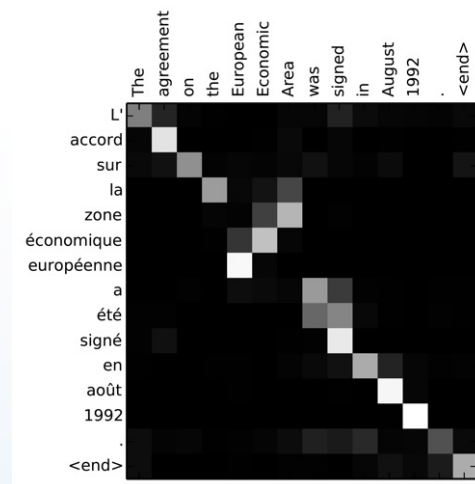
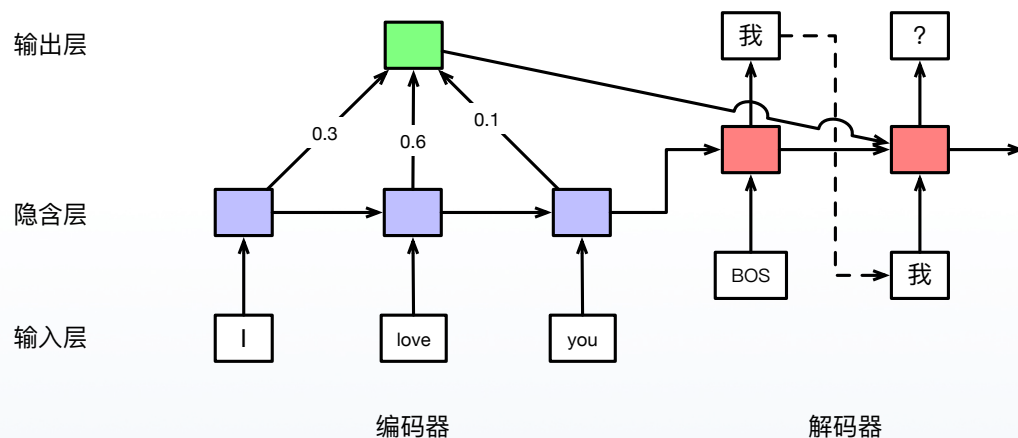
假设不合理

注意力模型



注意力机制 (Attention Mechanism)

□ 当前状态除和前一个状态及输入相关外，还应关注原序列的状态

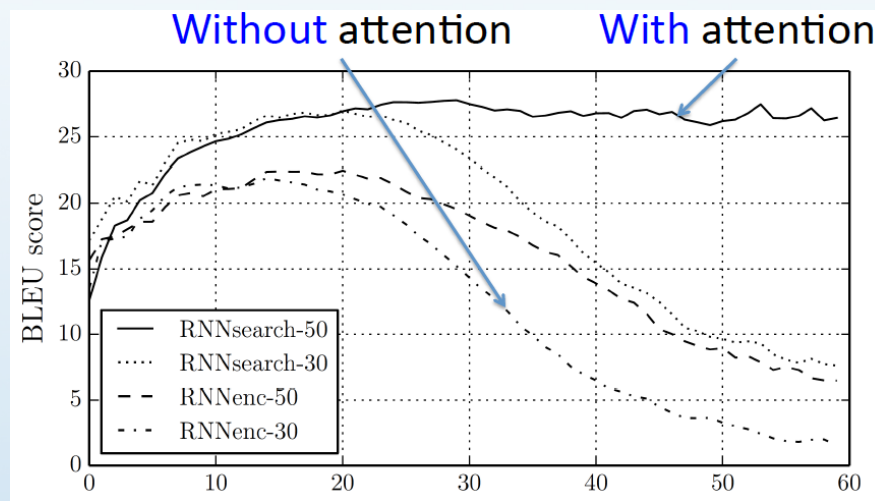


□ 打分公式

$$\hat{\alpha}_s = \text{attn}(\mathbf{h}_s, \mathbf{h}_{t-1})$$

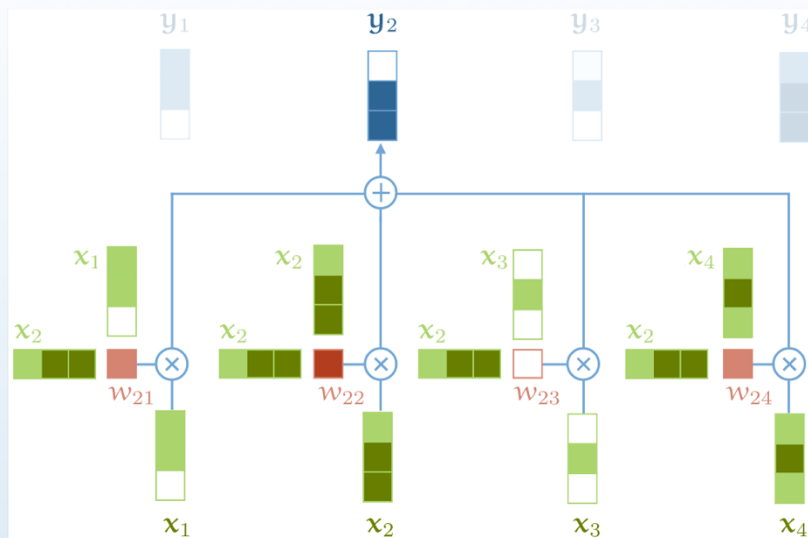
$$\alpha_s = \text{Softmax}(\hat{\alpha})_s$$

$$\text{attn}(\mathbf{q}, \mathbf{k}) = \begin{cases} \mathbf{w}^\top \tanh(\mathbf{W}[\mathbf{q}; \mathbf{k}]) \\ \mathbf{q}^\top \mathbf{W} \mathbf{k} \\ \mathbf{q}^\top \mathbf{k} \\ \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d}} \end{cases}$$



自注意力模型

- 通过某一输入与周围输入的相关性（注意力）来更新该输入
 - “观其伴、知其义”
- 输入： n 个向量构成的序列 $x_1, x_2, \dots, x_n$
- 输出： 每个向量对应的新向量 $y_1, y_2, \dots, y_n$
- $y_i = \sum_{j=1}^n \alpha_{ij} x_j$, α_{ij} 为 x_i 与 x_j 之间的注意力值



```
1 import torch
2 import torch.nn.functional as F
3
4 x = torch.rand(3, 4, 5)
5
6 # torch.bmm is a batched matrix multiplication. It
7 # applies matrix multiplication over batches of
8 # matrices.
9
10 raw_weights = torch.bmm(x, x.transpose(1, 2))
11
12 weights = F.softmax(raw_weights, dim=2)
13
14 y = torch.bmm(weights, x)
15
16 print(y)
```


Transformer

- Vaswani et al., **Attention** Is All You Need, NIPS 2017



- Transformer的翻译

- 变压器?
- 变形金刚?

- 自注意力模型还需要解决的几个问题

- 没有考虑输入的**位置**信息
- 输入向量 x_i 同时承担三种**角色**, 不易学习
 - 计算注意力权重时的两个向量以及被加权的向量
- 只考虑了两个输入向量之间的关系, 无法建模多个输入向量之间的**更复杂关系**



融入位置信息

□ 位置嵌入 (Position Embeddings)

- 类似词嵌入，每个绝对位置赋予一个连续、低维、稠密的向量表示
- 向量参数参与模型学习

□ 位置编码 (Position Encodings)

- 直接将一个整数 (位置索引值) 映射为一个向量

$$\text{PosEnc}(p, i) = \begin{cases} \sin\left(\frac{p}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{p}{10000^{\frac{i-1}{d}}}\right) & \text{else} \end{cases}$$

□ 输入向量 = 词向量 + 位置嵌入/编码

赋予输入向量角色信息

输入向量的三种角色

查询 (Query)

键 (Key)

值 (Value)

分别对输入向量进行线性映射

$$q_i = W^q x_i$$

$$k_i = W^k x_i$$

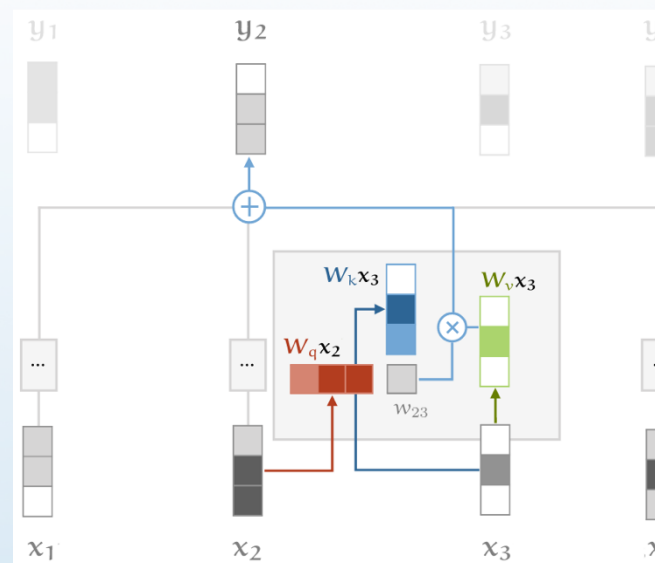
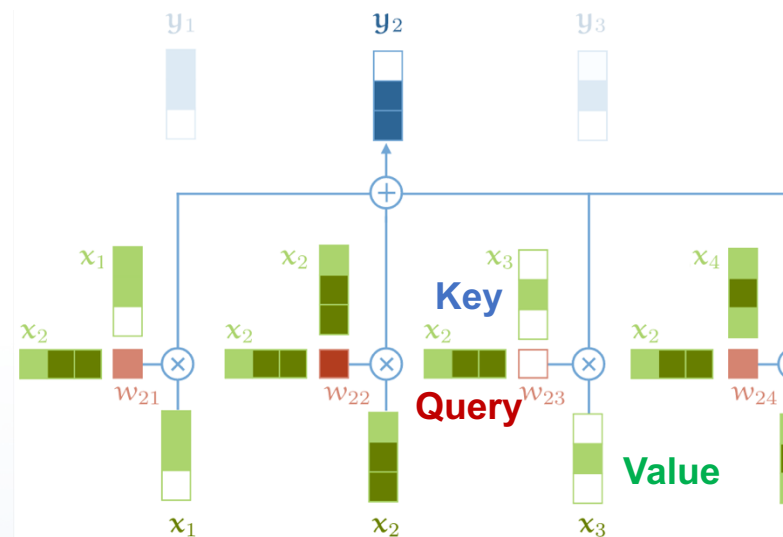
$$v_i = W^v x_i$$

新的自注意力模型

$$y_i = \sum_{j=1}^n \alpha_{ij} v_j$$

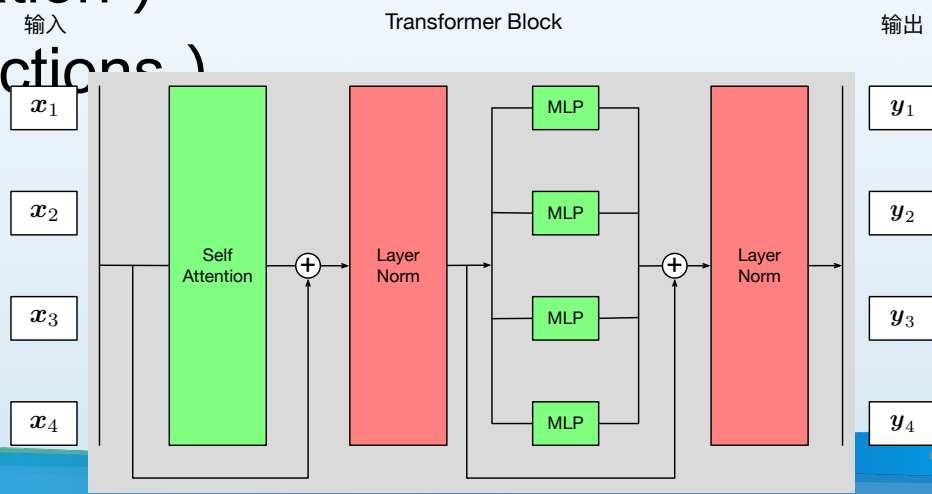
$$\alpha_{ij} = \text{Softmax}(\hat{\alpha}_{ij})$$

$$\hat{\alpha}_{ij} = \text{attn}(q_i, k_j)$$



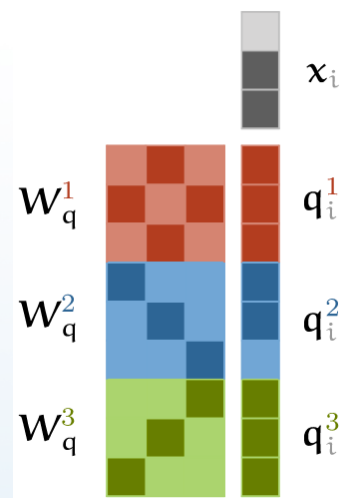
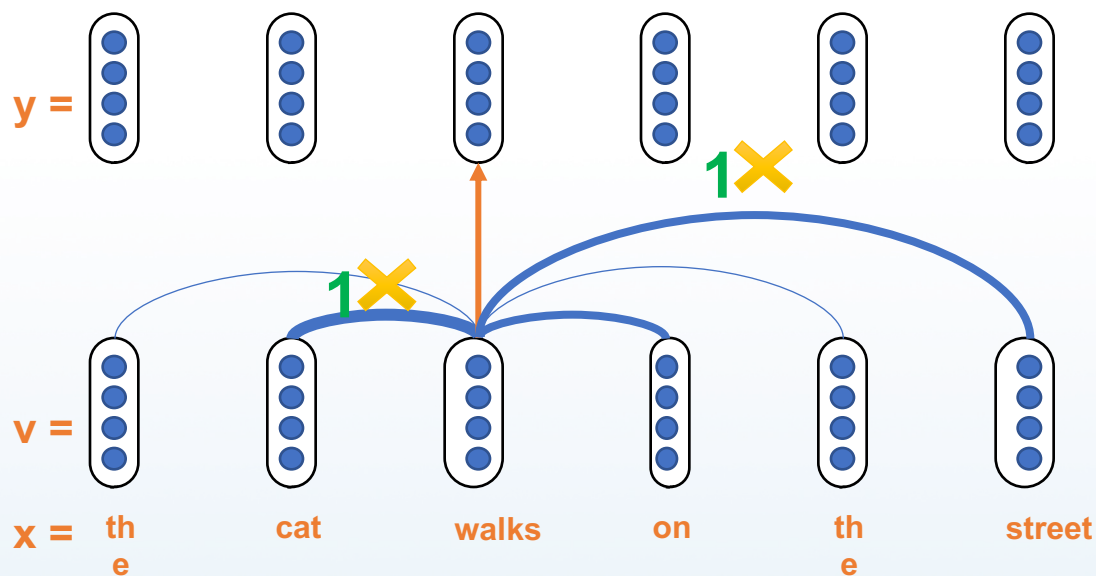
多层自注意力

- ❑ 原始自注意力模型仅考虑了任意两个向量之间的关系
- ❑ 如何建模高阶关系？
 - ❑ 直接建模高阶关系导致模型复杂度过高
 - ❑ 堆叠多层自注意力模型（消息传播机制）
- ❑ 增强模型的表示能力——增加非线性
 - ❑ 增加非线性的多层感知器模型（MLP）
- ❑ 使模型更容易学习
 - ❑ 层归一化（Layer Normalization）
 - ❑ 残差连接（Residual Connections）



多头自注意力

□ 由于Softmax函数的性质，无法使得多个自注意力分数趋近于1



□ 使用多组自注意力模型产生多组不同的注意力结果

□ 设置多组输入映射矩阵

□ 类似使用多个卷积核提取不同的特征

Transformer的优缺点

优点

- 直接建模**更长**距离的依赖关系
- **更快**的训练速度（与RNN相比）

缺点

- 参数量**过大**导致模型不容易训练
- 需要基于大规模数据**预训练**

Transformer模型实现

- ❑ 新版本的PyTorch(1.2版及以上)实现了Transformer模型
- ❑ nn.TransformerEncoder实现了编码模块
 - ❑ TransformerEncoderLayer多层堆叠构成
- ❑ nn.TransformerDecoder实现解码模块
 - ❑ TransformerDecoderLayer多层块堆叠构成

Transformer模型实现

```
>>> encoder_layer = nn.TransformerEncoderLayer(d_model=4, nhead=2)
    # 创建一个Transformer块，每个输入向量、输出的向量维度为4，头数为2
>>> src = torch.rand(2, 3, 4)
    # 随机生成输入，三个参数分别为序列的长度、批次的大小和每个输入向量的维度
>>> out = encoder_layer(src)
>>> print(out)
tensor([[[[-0.5909, -1.0048,  1.6249, -0.0293],
          [-1.7004,  0.5760,  0.2930,  0.8313],
          [-1.4910, -0.3054,  1.0853,  0.7111]],

        [[ 0.8265,  1.1358, -1.2065, -0.7557],
          [-0.2636,  0.0308,  1.5133, -1.2805],
          [-1.7299,  0.5828,  0.5041,  0.6430]]],
        grad_fn=<NativeLayerNormBackward>)
```

Transformer模型实现

□ 可以将多个Transformer块堆叠起来，构成一个完整的nn.Transformer Encoder

```
>>> transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)
>>> out = transformer_encoder(src)
>>> print(out)
tensor([[[[-0.0614, -0.7482,  1.6515, -0.8420],
          [-1.6981,  0.4108,  0.3998,  0.8875],
          [-1.6357,  0.3060,  1.0812,  0.2485]],

         [[-0.6341,  1.7177, -0.7156, -0.3680],
          [-1.3247,  1.3643,  0.4179, -0.4575],
          [-1.6706,  0.7775,  0.7674,  0.1258]]],
        grad_fn=<NativeLayerNormBackward>)
```


Transformer模型实现

□ decoder

```
>>> memory = transformer_encoder(src)
>>> decoder_layer = nn.TransformerDecoderLayer(d_model=4, nhead=2)
>>> transformer_decoder = nn.TransformerDecoder(decoder_layer, num_layers=6)
>>> out_part = torch.rand(2, 3, 4)
>>> out = transformer_decoder(out_part, memory)
>>> print(out)
```

```
tensor([[[[-0.0302, -0.4711,  1.6018, -1.1006],
          [ 0.0414, -0.3823,  1.5478, -1.2068],
          [-0.7133,  0.5378, -1.1745,  1.3500]],

        [[ 0.2694, -0.2363,  1.3747, -1.4077],
          [-0.1895,  0.1295,  1.4346, -1.3745],
          [-0.8469,  0.5927, -1.0769,  1.3311]]],

        grad_fn=<NativeLayerNormBackward>)
```


神经网络模型的训练

□ 寻找一组**优化**的模型参数

□ 又叫做模型**训练**或**学习**

□ 损失函数 (Loss Function)

□ 评估参数好坏的**准则**

□ 为什么不直接使用**准确率**等指标进行评估？

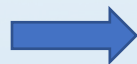
□ 两种常用的损失函数

□ **均方误差** (Mean Squared Error , MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

□ **交叉熵** (Cross-Entropy , CE)

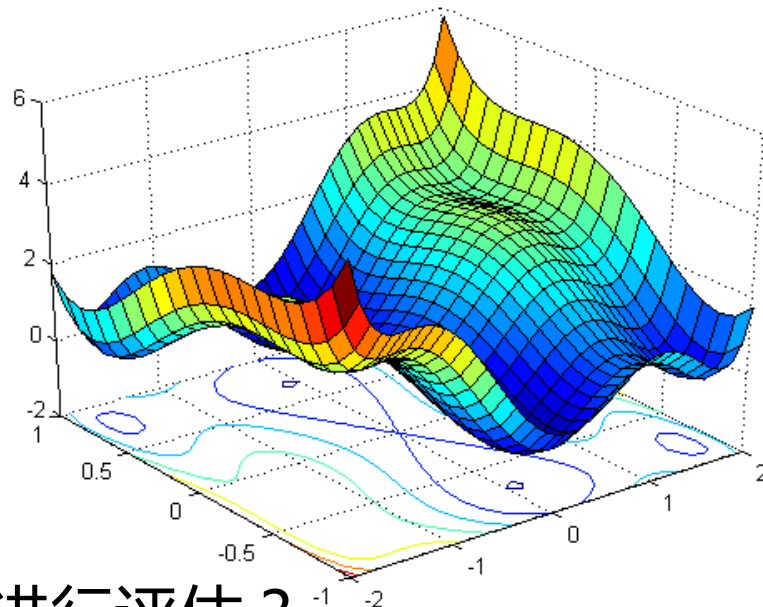
$$\text{CE} = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^c y_j^{(i)} \log \hat{y}_j^{(i)}$$



负对数似然损失

Negative Log Likelihood , NLL

$$\text{CE} = -\frac{1}{m} \sum_{i=1}^m \log \hat{y}_t^{(i)}$$



梯度下降

▣ 梯度 (Gradient)

- ▣ 以向量的形式写出的对多元函数各个参数求得的偏导数
- ▣ 是函数值增加最快的方向
- ▣ 沿着梯度相反的方向，更加容易找到函数的极小值

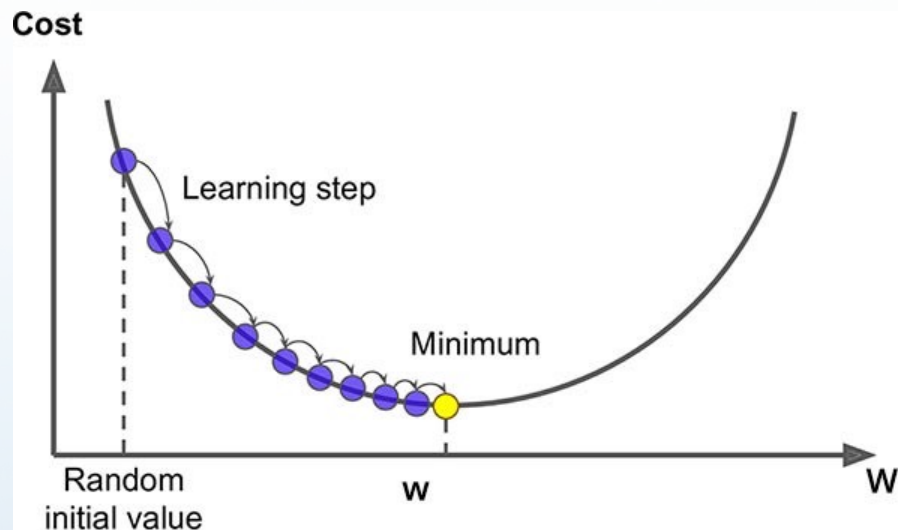
▣ 梯度下降算法 (Gradient Descent , GD)

算法 4.1 梯度下降算法

Input: 学习率 α ; 含有 m 个样本的训练数据

Output: 优化参数 θ

1. 设置损失函数为 $L(f(x; \theta), y)$;
2. 初始化参数 θ 。
3. **while** 未达到终止条件 **do**
4. 计算梯度 $g = \frac{1}{m} \nabla_{\theta} \sum_i^m L(f(x^{(i)}; \theta), y^{(i)})$;
5. $\theta = \theta - \alpha g$ 。
6. **end**



▣ 小批次梯度下降法 (Mini-batch Gradient Descent)

- ▣ 每次随机采样小规模的训练数据来估计梯度
- ▣ 提高算法的运行速度

PyTorch代码示例

```
# 创建多层感知器模型, 输入层大小为2, 隐含层大小为5, 输出层大小为2 (即有两个类别)
model = MLP(input_dim=2, hidden_dim=5, num_class=2)

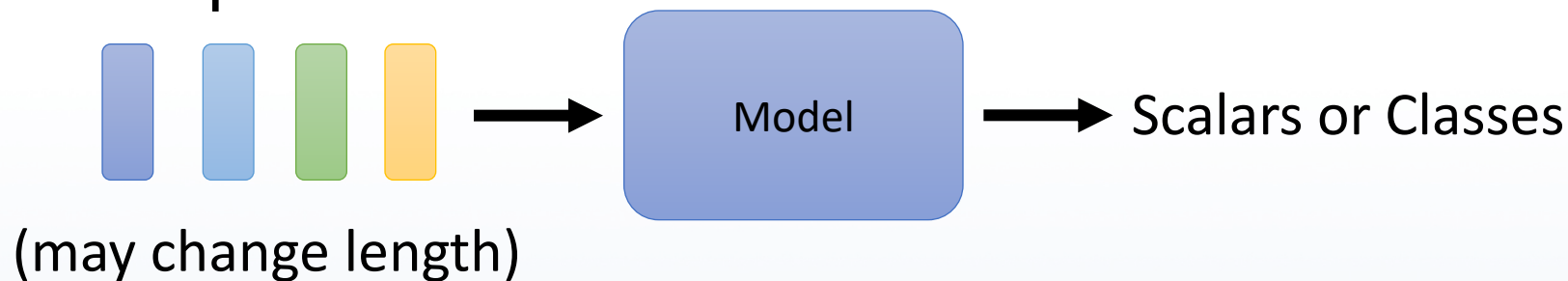
criterion = nn.NLLLoss() # 当使用log_softmax输出时, 需要调用负对数似然损失 (Negative Log Likelihood, NLL)
optimizer = optim.SGD(model.parameters(), lr=0.05) # 使用梯度下降参数优化方法, 学习率设置为0.05

for epoch in range(500):
    y_pred = model(x_train) # 调用模型, 预测输出结果
    loss = criterion(y_pred, y_train) # 通过对比预测结果与正确的结果, 计算损失
    optimizer.zero_grad() # 在调用反向传播算法之前, 将优化器的梯度值置为零, 否则每次循环的梯度将进行累加
    loss.backward() # 通过反向传播计算参数的梯度
    optimizer.step() # 在优化器中更新参数, 不同优化器更新的方法不同, 但是调用方式相同
```

情感分类为代表的文本分类任务

□ Input : is a set of vectors

□ Output:



情感分类为代表的序列标注任务

❑ 数据：NLTK 提供的句子倾向性分析数据 (sentence_polarity)

❑ 负面：

- ❑ simplistic , silly and tedious .
- ❑ it's so laddish and juvenile , only teenage boys could possibly find it funny . exploitative and largely devoid of the depth or sophistication that would make watching such a graphic treatment of the crimes bearable .

❑ 正面：

- ❑ the rock is destined to be the 21st century's new " conan " and that he's going to make a splash even greater than arnold schwarzenegger , jean-claud van damme or steven segal .
- ❑ the gorgeously elaborate continuation of " the lord of the rings " trilogy is so huge that a column of words cannot adequately describe co-writer/director peter jackson's expanded vision of j . r . r . tolkien's middle-earth .

名称

README.txt
rt-polarity.neg
rt-polarity.pos

情感分类为代表的文本分类任务

□ 大概过程：

- ① 数据处理
- ② 词表映射
- ③ 词向量层
- ④ 融入词向量层的多层感知器

情感分类为代表的文本分类任务

□ 大概过程：

① 数据处理

原始文本数据
↓
分句、标记解析等预处理

标记序列

simplistic , silly and tedious.



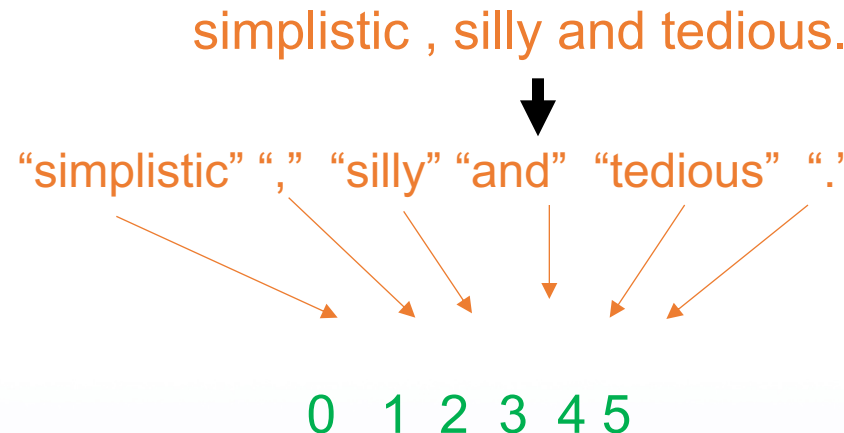
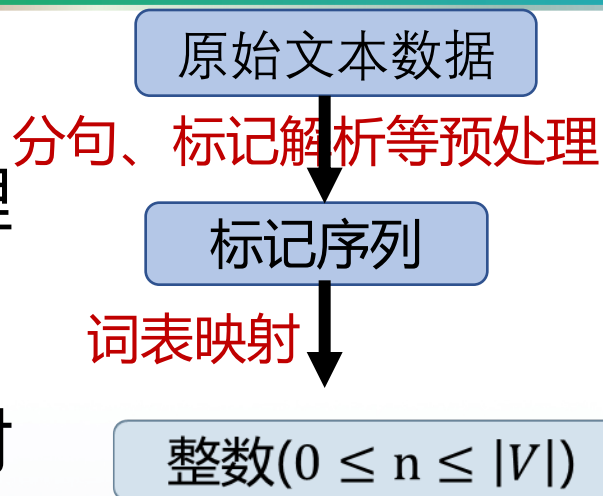
“simplistic” “,” “silly” “and” “tedious” “.”

情感分类为代表的文本分类任务

□ 大概过程：

① 数据处理

② 词表映射



情感分类为代表的文本分类任务

□ 大概过程：

① 数据处理

分句、标记解析等预处理

原始文本数据

标记序列

② 词表映射

词表映射

整数($0 \leq n \leq |V|$)

③ 词向量层

Embed-ding (低维、稠密、连续)

词向量

simplistic , silly and tedious.

"simplistic" " , "silly" "and" "tedious" "."

0 1 2 3 4 5

情感分类为代表的文本分类任务

大概过程：

① 数据处理

分句、标记解析等预处理

原始文本数据

标记序列

② 词表映射

词表映射

整数($0 \leq n \leq |V|$)

Embed-ding (低维、稠密、连续)

③ 词向量层

词向量

④ 融入词向量层的多层感知器

simplistic , silly and tedious.

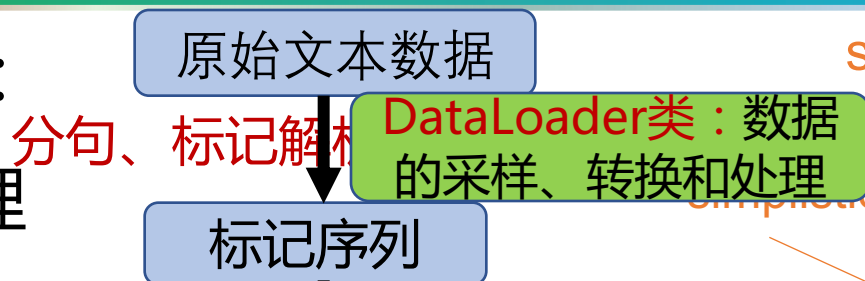
"simplistic" " ," "silly" "and" "tedious" "."

0 1 2 3 4 5

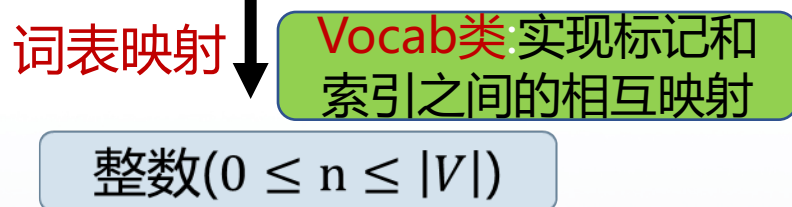
情感分类为代表的文本分类任务

大概过程：

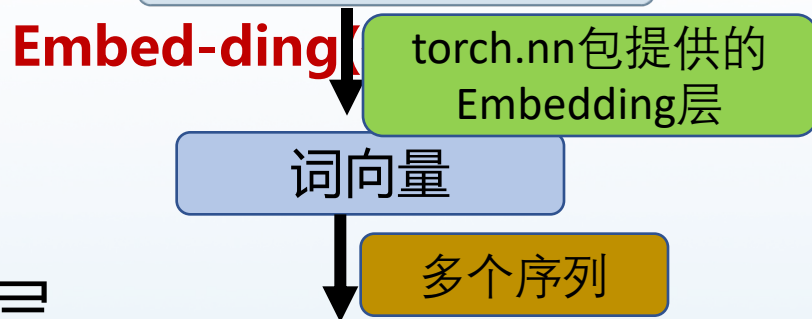
① 数据处理



② 词表映射



③ 词向量层



④ 融入词向量层的多层感知器

simplistic , silly and tedious.

simplistic " , " "silly" "and" "tedious" "."

0 1 2 3 4 5

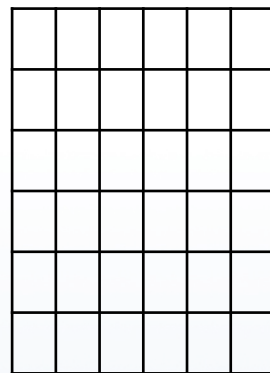
情感分类为代表的文本分类任务

□ 多词序列

□ 拼接($n \times d$): 预测结果与标记在序列中的位置过于相关

□ n : 词的个数

□ d : 向量的维数



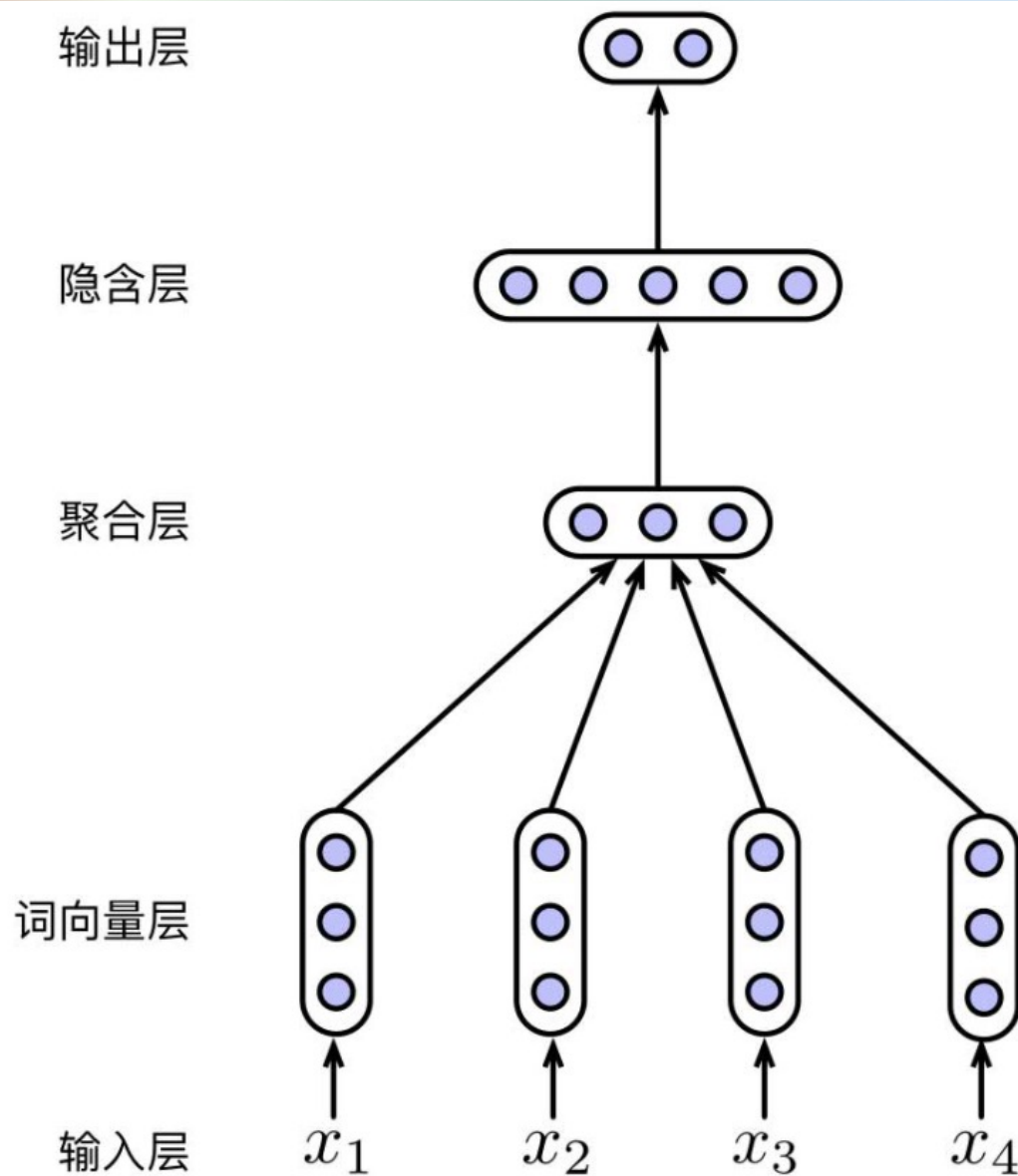
□ 词袋：聚合操作处理序列中的多个向量(求和、平均、最大)

□ 文本长度不一致：各个序列长度不一致，拼接的同时，用一个偏移量记录每个序列的其实位置

情感分类为代表的文本分类任务

```
>>> input1 = torch.tensor([0, 1, 2, 1], dtype=torch.long)
>>> input2 = torch.tensor([2, 1, 3, 7, 5], dtype=torch.long)
>>> input3 = torch.tensor([6, 4, 2], dtype=torch.long)
>>> input4 = torch.tensor([1, 3, 4, 3, 5, 7], dtype=torch.long)
>>> inputs = [input1, input2, input3, input4]
>>> offsets = [0] + [i.shape[0] for i in inputs]
>>> print(offsets)
[0, 4, 5, 3, 6]
>>> offsets = torch.tensor(offsets[:-1]).cumsum(dim=0)
>>> print(offsets)
tensor([ 0,  4,  9, 12])
>>> inputs = torch.cat(inputs)
>>> print(inputs)
tensor([0, 1, 2, 1, 2, 1, 3, 7, 5, 6, 4, 2, 1, 3, 4, 3, 5, 7])
```

情感分类为代表的文本分类任务



情感分类为代表的文本分类任务

❑ 基于卷积神经网络的情感分类

- ❑ 定义CNN的模型

- ❑ 重新定义整理函数:不同序列长度进行补齐

❑ 基于循环神经网络的情感分类

- ❑ 定义LSTM模型

- ❑ 不同的数据整理函数

❑ 基于Transformer的情感分类

- ❑ 定义Transformer模型

- ❑ 位置编码

- ❑ 数据输入

词性标注为代表的序列标注任务

□ 数据集：

- NLTK提供的宾州树库(Penn Treebank)的词性标注语料库

□ 基于循环神经网络的词性标注

□ 基于Transformer的词性标注

Thanks!

Q&A ?

□ wuxianyan@njau.edu.cn